



POLITECNICO
MILANO 1863

Design Document

Best Bike Paths Project

2025-2026

Deliverable:	DD
Title:	Design Document
Authors:	Andrea Ferritti, Jieke Lin, Tommaso Tosca
Version:	1.1
Date:	25-January-2025
Download page:	https://github.com/ferritti/FerrittiLinTosca
Copyright:	Copyright © 2025, A. Ferritti, J. Lin, T. Tosca – All rights reserved

Contents

Table of Contents	3
List of Figures	5
List of Tables	5
1 Document Information	7
2 Introduction	8
2.1 Purpose	8
2.1.1 Goals	8
2.2 Scope	8
2.3 Main Architectural Choices	8
2.4 Definitions, Acronyms, Abbreviations	9
2.4.1 Definitions	9
2.4.2 Acronyms	10
2.4.3 Abbreviations	10
2.5 Revision history	10
2.6 Reference Documents	10
2.7 Document Structure	10
3 Architectural Design	12
3.1 Overview	12
3.1.1 System View	12
3.1.2 Detailed View	13
3.2 Component View	14
3.2.1 ClientApplication Component	14
3.2.2 WebServer Component	14
3.2.3 ExternalProxy component	15
3.2.4 ApplicationServer Component	15
3.2.5 Data Component	18
3.2.6 External Services	18
3.3 Deployment View	18
3.4 Runtime View	19
3.5 Component Interface	29
3.5.1 AccountManager Interfaces	29
3.5.2 PathManager Interfaces	29
3.5.3 TripManager Interfaces	30
3.5.4 ReportManager Interfaces	30
3.5.5 DataManager Interfaces	30
3.5.6 Database Interfaces	32
3.5.7 WebServer Interfaces	32
3.5.8 ExternalProxy Interfaces	33
3.5.9 ClientApplication Interfaces	34
3.5.10 MapService Interfaces	35
3.5.11 WeatherService Interfaces	35
3.5.12 Sensors Interfaces	36
3.6 Selected Architectural Styles and Patterns	36
3.7 Other Design Decisions	36

4	User Interface Design	38
5	Requirements Traceability	43
6	Implementation, Integration, and Test Plan	52
6.1	Plan Overview	52
6.2	Implementation Plan	53
6.3	Component Integration and Testing	54
6.4	System Testing	58
7	Use of Generative AI	59
7.1	Example of Interaction: Publish Path Sequence Diagram Description	59
7.2	Conclusion	60
8	Effort Spent	61

List of Figures

1	Four-tier architecture	12
2	Component Diagram	14
3	Account Manager	15
4	Path Manager	16
5	Trip Manager	17
6	Deployment Diagram	18
7	[SD1] Signup Sequence Diagram	19
8	[SD2] Login Sequence Diagram	20
9	[SD3] Search Path Sequence Diagram	21
10	[SD4] Start Navigation Sequence Diagram	22
11	[SD5] Save Trip Sequence Diagram	23
12	[SD6] View Trips Sequence Diagram	24
13	[SD7] Edit Profile Sequence Diagram	25
14	[SD8] Delete Path Sequence Diagram	25
15	[SD9] Publish Path Sequence Diagram	26
16	[SD10] Automated Mode Sequence Diagram	27
17	[SD11] Publish Recorded Path Sequence Diagram	28
18	Home page	38
19	Profile screen	39
20	Edit profile screen	40
21	Login screen	41
22	SignUp screen	42
23	Dependencies Diagram	52
24	Stage 1	54
25	Stage 2	55
26	Stage 3	56
27	Stage 4	57

List of Tables

1	Requirement R1 - Traceability	44
2	Requirement R2 - Traceability	44
3	Requirement R3 - Traceability	44
4	Requirement R4 - Traceability	44
5	Requirement R5- Traceability	45
6	Requirement R6 - Traceability	45
7	Requirement R7 - Traceability	45
8	Requirement R8 - Traceability	45
9	Requirement R9 - Traceability	46
10	Requirement R10 - Traceability	46
11	Requirement R11 - Traceability	46
12	Requirement R12 - Traceability	46
13	Requirement R13 - Traceability	47
14	Requirement R14 - Traceability	47
15	Requirement R15 - Traceability	47
16	Requirement R16 - Traceability	47
17	Requirement R17 - Traceability	48
18	Requirement R18 - Traceability	48

19	Requirement R19 - Traceability	48
20	Requirement R20 - Traceability	48
21	Requirement R21 - Traceability	49
22	Requirement R22 - Traceability	49
23	Requirement R23 - Traceability	49
24	Requirement R24 - Traceability	49
25	Requirement R25 - Traceability	50
26	Requirement R26 - Traceability	50
27	Requirement R27 - Traceability	50
28	Requirements Traceability Matrix	51
29	Effort - Andrea Ferritti	61
30	Effort - Jieke Lin	61
31	Effort - Tommaso Tosca	61

1 Document Information

Version	Modification Details
v1.0 7-01-2025	First version.
v1.1 25-01-2025	Modifications: • The interaction with the Map Service has been added in SD9 and SD10 to retrieve street names; • The chooseStatus interaction and its corresponding response have been added in SD9; • In SD5, the request name chooseStatus was changed to changeStatus and it was placed inside an opt block; • In SD4, the parameter weatherInfo was added to mostFrequentWeather().

2 Introduction

2.1 Purpose

The purpose of the Best Bike Paths (BBP) system is to provide a software architecture that enables cyclists to record their trips, creating a personal history of their activities, while simultaneously building a shared, community-driven inventory of bike paths. BBP allows registered users to insert information about path quality and obstacles either manually or through an automated data collection process that leverages mobile device sensors (GPS, accelerometer, gyroscope). Furthermore, the system processes this distributed data to calculate path scores, merging conflicting reports based on freshness and consensus, to allow any user to visualize the safest and most effective paths on a map. The reader can find more information about the problem in the RASD document. In the remaining part of this chapter and the subsequent sections, we will present a summary of the technical choices made for the creation of the platform. This includes the definition of the architectural style, the decomposition into high-level components, and the design of the user interfaces. We also include the specific Goals we are trying to accomplish with this design and the Definitions, Acronyms, and Abbreviations used throughout this document.

2.1.1 Goals

- [G1] Registered users would like to maintain a history of their trips.
- [G2] Registered users would like to access performance metrics about their trips.
- [G3] When available, registered users would like to be informed about meteorological information related to their trips.
- [G4] Registered users would like to share information about specific bike paths.
- [G5] Users would like to find the optimal bike path between an origin and a destination by comparing viable alternatives based on efficiency and status.
- [G6] Users would like to access reliable and up-to-date shared information about bike paths.

2.2 Scope

The scope of this Design Document is to translate the requirements previously defined in the RASD into a concrete software architecture for the Best Bike Paths (BBP) system. This document provides a comprehensive technical overview of the system, focusing on the decomposition of high-level components, the design of internal and external interfaces, and the rationale behind the selected architectural patterns. Beyond the structural design, this document establishes the guidelines for the subsequent development phases. It defines the Implementation, Integration, and Test Plan, outlining the methodologies and tools that will be employed to ensure the correct functioning of the platform.

2.3 Main Architectural Choices

The system follows a four-tier architectural style, logically organized into presentation tier, web tier, business tier and data tier. The presentation tier is responsible for the user interface and the interaction with the platform, providing a responsive experience for visualizing paths and managing cycling activities. It also interact with the map service to get the map and the graphical libraries. Requests from the client are handled by the web tier, where a web server acts as the primary coordinator and forwards them to the business tier, which is responsible for the platform's logic. The web tier also communicates with the weather service through an external proxy. The data tier ensures the persistence and integrity of all

system information through dedicated Database Management Systems (DBMS), and to maximize security, the data tier is hosted within a private network, accessible only by the business tier through secure, internal connections. This architectural choice allows high scalability, maintainability, and security. The use of a web server and a dedicated proxy centralizes traffic control and security policies, shielding the core logic and sensitive data from direct exposure to the public internet. Furthermore, this separation of concerns ensures that the platform remains flexible and highly available through component replication, allowing developers to update the user interface or integrate new external services without compromising the stability of the underlying data and business processes.

2.4 Definitions, Acronyms, Abbreviations

The following section provides a comprehensive list of definitions, acronyms, and abbreviations used throughout this document. Its purpose is to assist the reader in understanding specific technical terminology and domain-specific concepts adopted in the design of the BBP system.

2.4.1 Definitions

- **Bike Path:** A road segment or sequence of segments that is suitable for bicycles. A path is considered bike-safe if it includes a dedicated bike lane or if car traffic is rare and speed limits are compatible with typical bike speeds.
- **Trip:** A cycling activity recorded by a registered user. A trip includes GPS tracking data, performance metrics (distance, average speed, etc.), and, when available, meteorological information retrieved from an external weather service.
- **Performance Metrics:** Statistics associated with a trip, such as total distance, duration, average speed, maximum speed and other cycling-related measures.
- **Meteorological Information:** External data enriched by BBP when available, including temperature, weather conditions (sunny, cloudy, rainy...), wind speed, humidity, and similar environmental indicators.
- **Manual Mode:** A mode in which a user manually enters information about a bike path, specifying street names, path segments, their status, and any obstacles.
- **Automated Mode:** A mode in which BBP automatically collects data from the user's mobile device while biking. Data includes GPS traces to infer the following path, accelerometer and gyroscope readings to detect potential potholes or obstacles, and speed information to infer that the user is cycling.
- **Obstacle:** Any relevant physical irregularity affecting the bike path (e.g., potholes, bumps, debris). In automated mode, obstacles are inferred from device sensors and later confirmed by the user.
- **Status:** A qualitative assessment of a bike path segment. Possible values include: optimal, medium, sufficient, requires maintenance, etc. Status can be user-generated (manual or automatic) and may be aggregated from multiple users.
- **Information Publishable:** Verified and user-approved data about a bike path (status, obstacles) that can be made publicly accessible to the community.
- **Score:** A computed score used to rank alternative bike paths between an origin and a destination. The score considers both the quality (status) of the path and its effectiveness in connecting the two points.

- **External proxy:** An external proxy server acts as an intermediary between a client device and the Internet. It intercepts outgoing HTTP/HTTPS requests, forwards them to external services using its own IP address, thereby masking the client's IP, and relays the corresponding responses back to the client.
- **4-tier architecture:** A software architecture that divides the software into four different tiers: presentation tier, web tier, application tier, and data tier.
- **Query:** A database query is a request to retrieve or update data that meets a specific search criterion.

2.4.2 Acronyms

- **BBP:** Best Bike Path.
- **GPS:** Global Positioning System.
- **API:** Application Programming Interface.
- **DMZ:** Demilitarized zone.
- **RASD:** Requirements Analysis and Specification Document.
- **HTTPS:** Hypertext Transfer Protocol Secure.

2.4.3 Abbreviations

- **SD:** Sequence diagram.

2.5 Revision history

- Version 1.0: 7/01/2026
- Version 1.1: 25/01/2026

2.6 Reference Documents

- Assignment RDD AY 2025-2026
- Software Engineering 2 AY 2025/2026 slides "CreatingDD"

2.7 Document Structure

1. **Introduction:** This chapter sets the context for the design process. It outlines the purpose and scope of the document, lists the definitions and acronyms essential for understanding the technical terminology, and references the materials that served as a baseline for this work.
2. **Architectural Design:** This is the core of the document, offering a comprehensive breakdown of the BBP system. It starts with a high-level overview of the macro-components and their relationships, then drills down into a detailed description of individual modules and their interfaces. This section also rationalizes the choice of specific architectural styles and patterns, concluding with the deployment strategy and runtime views that illustrate the system's dynamic behavior.
3. **User Interface Design:** This section focuses on the interaction layer of the application. Based on the descriptions provided in the RASD document, it presents the visual layout of the system through mockups.

4. **Requirements Traceability:** To ensure complete coverage of the project specifications, this chapter maps the functional requirements defined in the RASD to the specific design components and modules described in this document.
5. **Implementation, Integration, and Test Plan:** This chapter defines the roadmap for the construction of the system. It specifies the sequence in which components will be implemented and integrated, and details the testing strategies and tools that will be employed to verify the software's correctness and quality.
6. **Use of Generative AI:** In this chapter, examples of interactions are provided to explain how AI was used to support the drafting of the document.
7. **Effort Spent:** It reports the time spent to produce this document, organized by section.

3 Architectural Design

3.1 Overview

This chapter outlines the four-layered architecture of the Best Bike Paths (BBP) system. We begin by mapping the high-level tiers and their primary communication paths. This is followed by an exploration of the individual components and their specific functional responsibilities. The document then describes the deployment strategy and the runtime dynamics between components, including a formal definition of their interfaces. Finally, we discuss the design patterns and strategic choices implemented to ensure the system meets its scalability and fault-tolerance objectives.

3.1.1 System View

The Best Bike Paths (BBP) platform is organized into a four-tier architecture consisting of a presentation tier, web-tier, business-tier and a dedicated data tier. The presentation tier handles user interactions, sensor data acquisition and communication with the map service, while the web tier handle the interaction between the presentation and the business tier, and it includes an external proxy for API calls to the weather service. The web server manages HTTPS requests from the presentation tier and routing internal requests to the business tier, while the business tier hosts the core logic of the application, and the data tier includes the database server. To ensure high availability and resilience, all components are designed for replication and container-based deployment, effectively preventing single points of failure across the system.

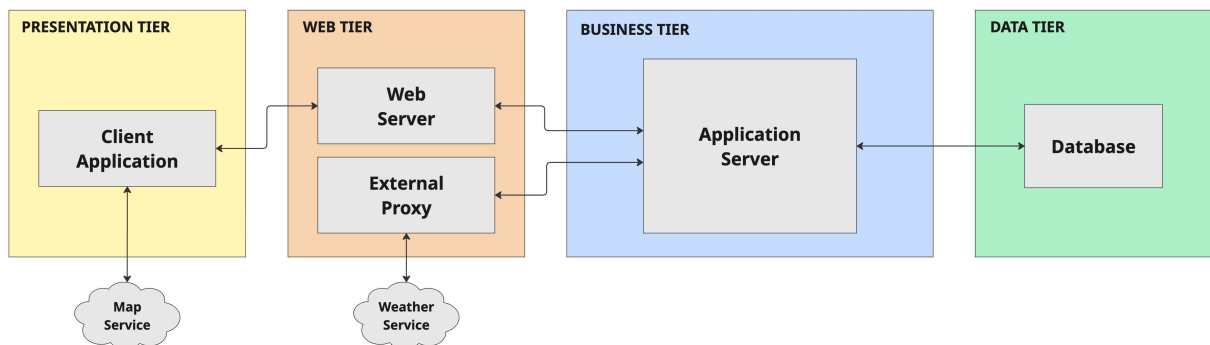


Figure 1: Four-tier architecture

3.1.2 Detailed View

Presentation Tier

The presentation tier of Best Bike Paths (BBP) consists entirely of the interface accessible directly through the user's device. This layer is responsible for managing human–computer interaction, transforming the cyclist's actions such as viewing a map or analyzing a path into asynchronous requests to the web tier, and to extract data from the sensors and send them, through the web tier, to the application tier. It also interact with the map service to get the map and the graphical libraries to draw on it.

Web Tier

The web tier serves as the primary coordination and security layer, and represents the demilitarized zone (DMZ) of the system. It hosts a web server that acts as a reverse proxy, receiving incoming requests from the presentation tier and routing them to the business tier. It also integrates a load balancer to optimize and distribute traffic efficiently across the system. Additionally, this layer includes an external proxy to handle outgoing interactions with the weather service, keeping the business tier isolated from the public network. This centralized approach enables efficient load balancing and ensures that all traffic is strictly monitored and secured before reaching the platform's internal components.

Business Tier

The business tier constitutes the logical core of the system, where all the main functionalities of the Best Bike Paths platform are processed. It integrates a load balancer to manage the data distribution across the replicas. Operating within an isolated network, it communicates with the data tier for information persistence through queries. Thanks to its replicated and containerized structure, the tier ensures high scalability and resilience in handling computational workloads.

Data Tier

The data tier is dedicated to the long-term persistence of all system information, managed through DBMS. It resides in a private network completely isolated from the outside, being accessible exclusively by the business tier to ensure maximum protection of sensitive data. Its replicated structure guarantees the integrity of the information and high availability of the service.

3.2 Component View

This section presents the Component View of the Best Bike Paths system. It describes the main software components composing each architectural layer and their responsibilities, along with the interfaces through which they interact. The description follows a top-down approach, starting from the most general components and progressively refining the analysis by focusing on the most relevant and complex ones. WeatherServiceI and MapServiceI represent the interfaces of the external services required for the correct functioning of the system

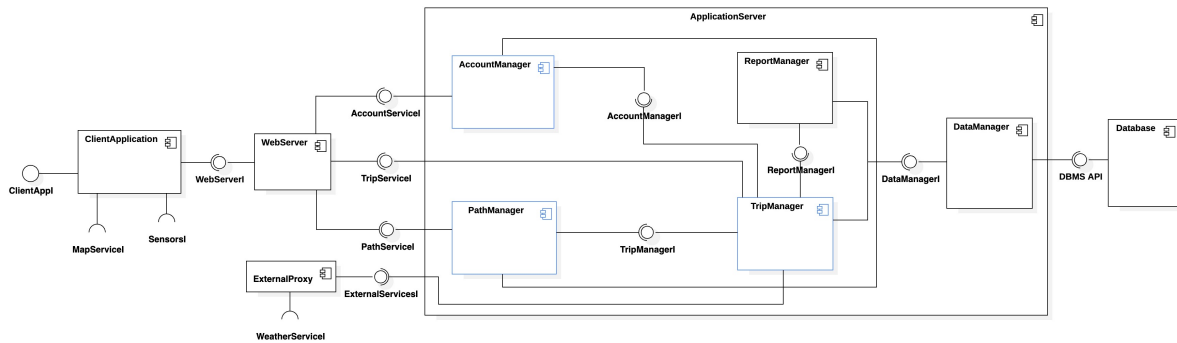


Figure 2: Component Diagram

3.2.1 ClientApplication Component

The ClientApplication Component serves as the Presentation Tier of the application. Its primary function is to render the graphical interfaces enabling end-users to interact with the system's features and services. The component's internal logic is primarily dedicated to interface management, such as preliminary data validation and real-time user feedback. This component is designed for on-road use, providing a portable interface optimized for the cycling experience. It integrates with device hardware to support real-time trip tracking and path-surface monitoring. It offers the complete set of system features in a format that is easy to interact with while on the move, ensuring cyclists have access to all tools during their journeys.

3.2.2 WebServer Component

The WebServer component acts as the primary entry point for all incoming traffic from the Presentation Tier. Located within the Web Tier, it handles the reception of HTTPS requests and the logical routing of requests toward the appropriate components of the Business Tier. To ensure scalability and high availability, the Web Server is deployed in multiple replicated instances behind a load balancer. Incoming requests are distributed among these replicas, allowing the system to handle higher traffic volumes and to remain operational in case of individual instance failures.

3.2.3 ExternalProxy component

The ExternalProxy, located within the Web Tier, is the component dedicated to the centralized management of all outgoing (outbound) communications to third-party service providers. It acts as an intermediary between the Business Tier and external APIs of weather service, required to enrich path information. This separation ensures that the Business Tier remains isolated from the public network, delegating to the proxy the task of interacting with the external world in a secure and controlled manner.

3.2.4 ApplicationServer Component

The ApplicationServer, located in the Business Tier, is responsible for executing the entire application logic, processing validated requests coming from the Web Tier. Operating within an isolated private network, it communicates with the Data Tier for information persistence and with the ExternalProxy for retrieving weather data, ensuring high scalability and resilience through a replicated and containerized structure. It is divided into the following components:

- **AccountManager**

The Account Manager component handles all operations related to user identity, security, and profile evolution. It exposes the *AccountServiceI* interface for external interactions and the *AccountManagerI* interface to provide user-related data to other internal components. It communicates with the Data Manager through the *DataManagerI* interface to ensure the persistence and integrity of user credentials and statistics.

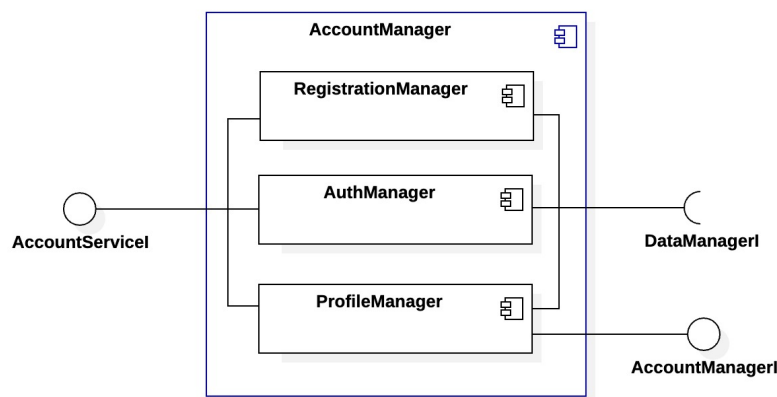


Figure 3: Account Manager

1. **RegistrationManager**

It manages the cyclists registration. It is responsible for validating user input, ensuring the uniqueness of emails, and initializing basic performance statistics. It interacts with the *DataManagerI* to store new profile records.

2. **AuthManager**

Represents the core of system security. It manages session tokens, password encryption, and access permissions, ensuring that every sensitive operation is strictly authorized.

3. **ProfileManager**

Administers the cyclist's profile and handles the modification of personal data. It tracks cumulative performance metrics, specifically the total distance traveled and global average speed, interfacing with the persistence layer to provide an up-to-date summary of the user's activity.

- **PathManager**

The Path Manager handles path's lifecycle. It provides the *PathServiceI* to allow the WebServer to send request from the ClientApplication. It relies on the *DataManagerI* for path data and the *TripManagerI* to save a trip.

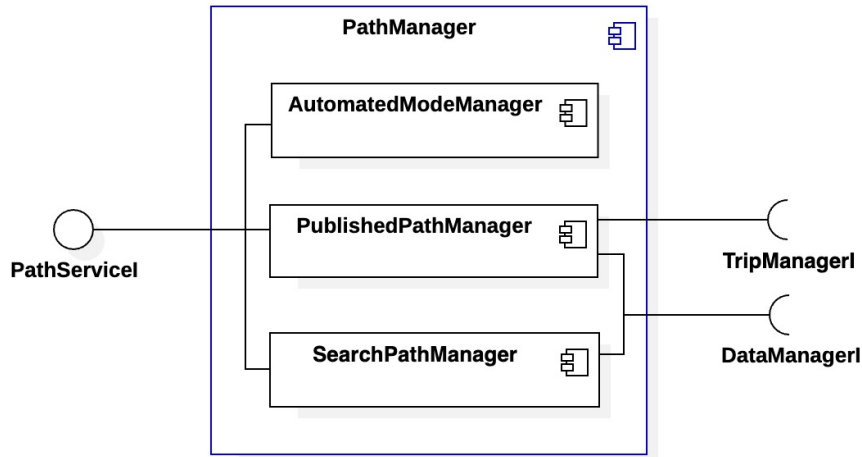


Figure 4: Path Manager

1. **AutomatedModeManager**

Manages the passive tracking logic. When a user begins a trip without a pre-set destination, this sub-component analyzes movements and reconstructs the path in real-time.

2. **PublishedPathManager**

It handles path management and the manual mode.

3. **SearchPathManager**

Provides the filtering engines. It allows the path database to be queried by providing origin and destination of path of interest, and returns .

- **TripManager**

The Trip Manager provides the *TripServiceI* to allow users to visualize the trip history and manage the trip navigation, and the *TripManagerI* interface to manage the trip history. It requests services from the Account Manager (*AccountManagerI*) to make account's data consistent, from the Report Manager (*ReportManagerI*) to report obstacle and status updates, and from the Data Manager (*DataManagerI*) for history storage. It uses *ExternalServicesI* to access weather services.

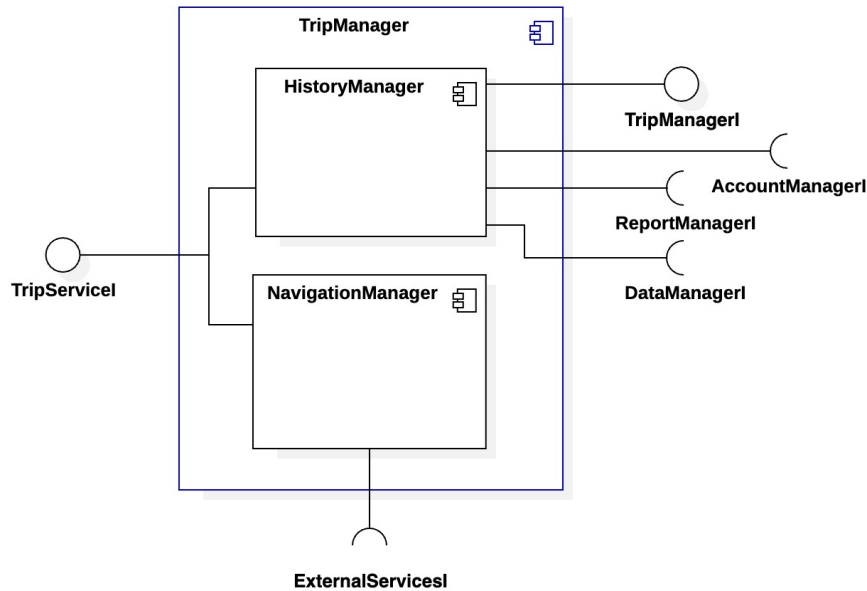


Figure 5: Trip Manager

1. **NavigationManager**

Processes GPS data to provide guidance instructions and compares the current location against the planned path.

2. **HistoryManager**

Once a session is finalized, it consolidates all trip data including average speed and total distance. It then invokes the *DataManagerI* to archive the trip for later viewing on the dashboard.

- **ReportManager**

The Report Manager exposes the *ReportManagerI* interface to receive real-time alerts and status updates from the Trip Manager and uses the *DataManagerI* to store all reported information. This component collects reports originating from navigations, whose information can be modified manually at the end of the navigation. Reports concerning the same path, collected within the same timeframe, are merged to update path status and obstacle detection within the PathManager. This establishes a virtuous cycle where every user contributes to collective safety.

3.2.5 Data Component

The Database is an external component dedicated to the secure and permanent storage of all system information.

- **Role:** It serves as the central repository for sensitive user profiles, the library of published bike paths, historical trips, and reported path obstacles and status.
- **Interaction:** It communicates exclusively with the DataManager via the DBMS API interface. This separation ensures data integrity and optimized CRUD operations.

3.2.6 External Services

- **Map Service**
Provides the foundational geographic data. It offers detailed road network information, enabling the system to visualize paths and guide users.
- **Weather Service**
Delivers real-time atmospheric telemetry, including temperature, wind speed, and current conditions.

3.3 Deployment View

The Deployment View describes the physical mapping of software components onto hardware nodes and the network infrastructure, illustrating the system's distribution and defining its container organization. The following figure shows the platform's deployment diagram.

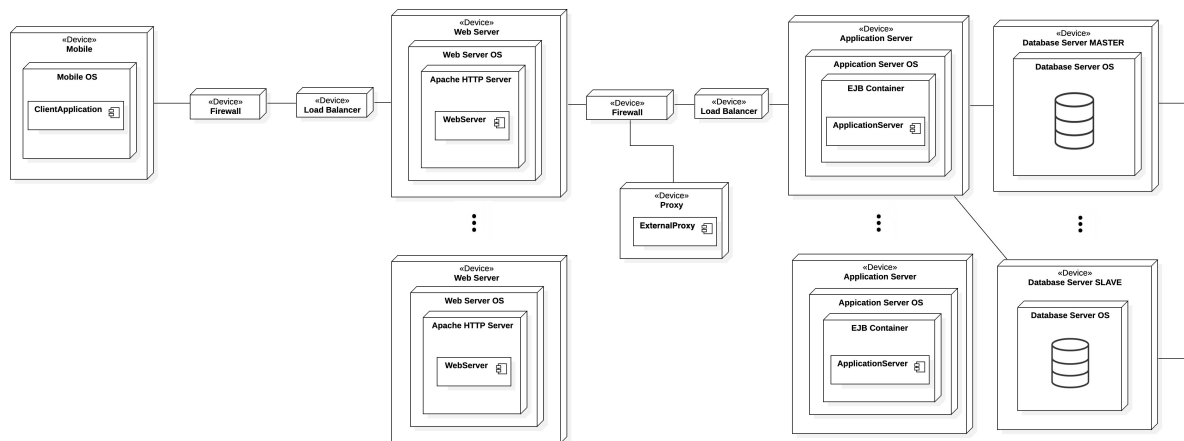


Figure 6: Deployment Diagram

This Deployment View illustrates a four-layer architecture designed to ensure scalability and security through a clear separation of responsibilities. Incoming traffic from the presentation tier first passes through a firewall and is then distributed by an initial load balancer among multiple replicas of the web server, optimizing request handling and service availability. Similarly, calls to the business logic pass through another firewall and are then handled by a second load balancer, which distributes the load across the ApplicationServer replicas. The data tier adopts a master–slave architecture in which the ApplicationServer and its replicas read from and write to the master, which continuously updates the slaves. In the case of a master failure, the ApplicationServer and its replicas read data from the slaves. Outbound calls to weather service pass through the proxy, preventing the direct exposure of the application server's IP address. In this case, the responses received from the weather service pass

through the firewall which, unlike the data flow coming from the web server, forwards them directly to the application server without passing through the load balancer.

3.4 Runtime View

This section describes the dynamic behavior of the system during its execution, illustrating how components interact with each other to fulfill the functional requirements. Unlike static views, this section focuses on control flows, message exchanges, and the sequencing of operations among the various components defined in the component view. Through the analysis of the use cases defined in the RASD document, it enables a clear understanding of the request lifecycle and the coordination of processes in real time.

[SD1] Signup Sequence Diagram

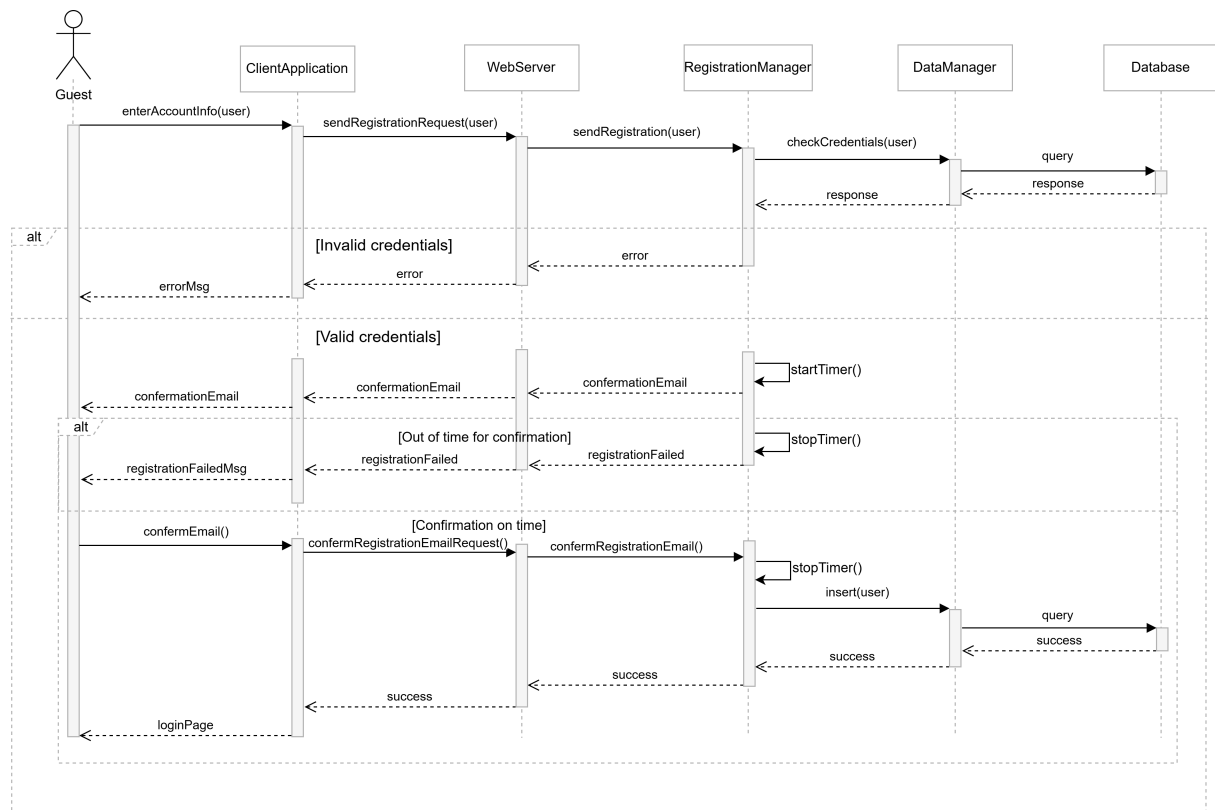


Figure 7: [SD1] Signup Sequence Diagram

This sequence diagram describes the registration flow. The RegistrationManager first validates the submitted credentials; if valid, it initiates a time-bound verification process by sending a confirmation email and starting an internal timer.

The process has two possible outcomes based on user action:

- The operation aborts if the initial credentials are invalid or if the user fails to confirm the email before the timer expires.
- The system permanently inserts the new user into the Database only if the Guest confirms the registration within the validity window.

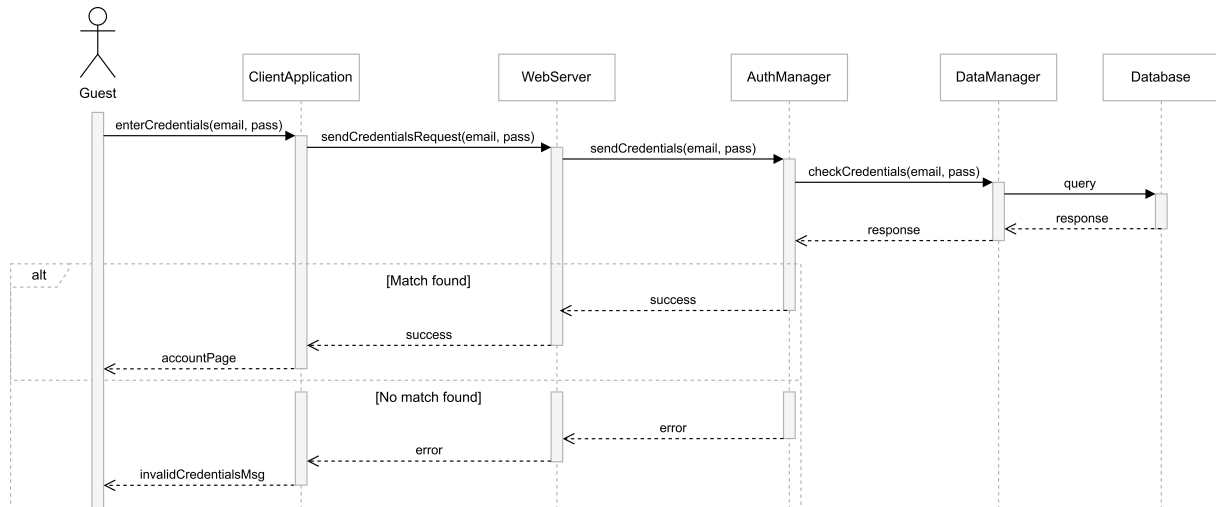
[SD2] Login Sequence Diagram

Figure 8: [SD2] Login Sequence Diagram

This sequence diagram describes the login process of a guest. The guest enters their email and password in the ClientApplication, which forwards the credentials to the WebServer. The WebServer passes them to the AuthManager, which asks the DataManager to verify the credentials by querying the Database.

- If the credentials match (match found), the successful result propagates back through the chain and the guest is redirected to the account page.
- If no match is found, an error is returned and the ClientApplication displays an invalidCredentials message to the guest trip.

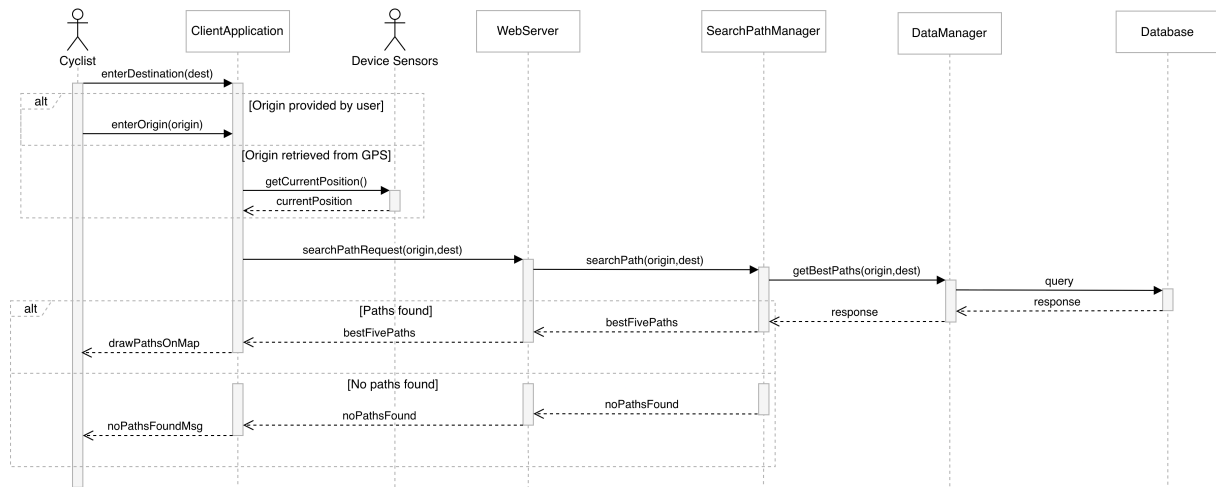
[SD3] Search Path Sequence Diagram

Figure 9: [SD3] Search Path Sequence Diagram

This sequence diagram describes the path search process initiated by a cyclist. It is assumed that, upon opening the application, the cyclist is on the home page and the map has already been loaded by the SearchPathManager through the MapService. The interaction begins with the Cyclist entering the destination in the ClientApplication. An alternative flow determines the origin: it is either manually provided by the user (enterOrigin) or automatically retrieved from the Device Sensors via the GPS (getCurrentPosition). Once the origin and destination are established, the ClientApplication sends a request to the WebServer. The WebServer forwards the request to the SearchPathManager, which is responsible for the search logic. The SearchPathManager then contacts the DataManager, which interacts with the Database to query for the best paths.

- If paths are found, the bestFivePaths data is propagated back through the system (via the SearchPathManager and WebServer) to the ClientApplication, which then performs the drawPathsOnMap action to visualize the paths for the cyclist.
- Otherwise, if no suitable paths are found, a noPathsFound signal is returned through the layers, and a specific message (noPathsFoundMsg) is displayed to inform the cyclist.

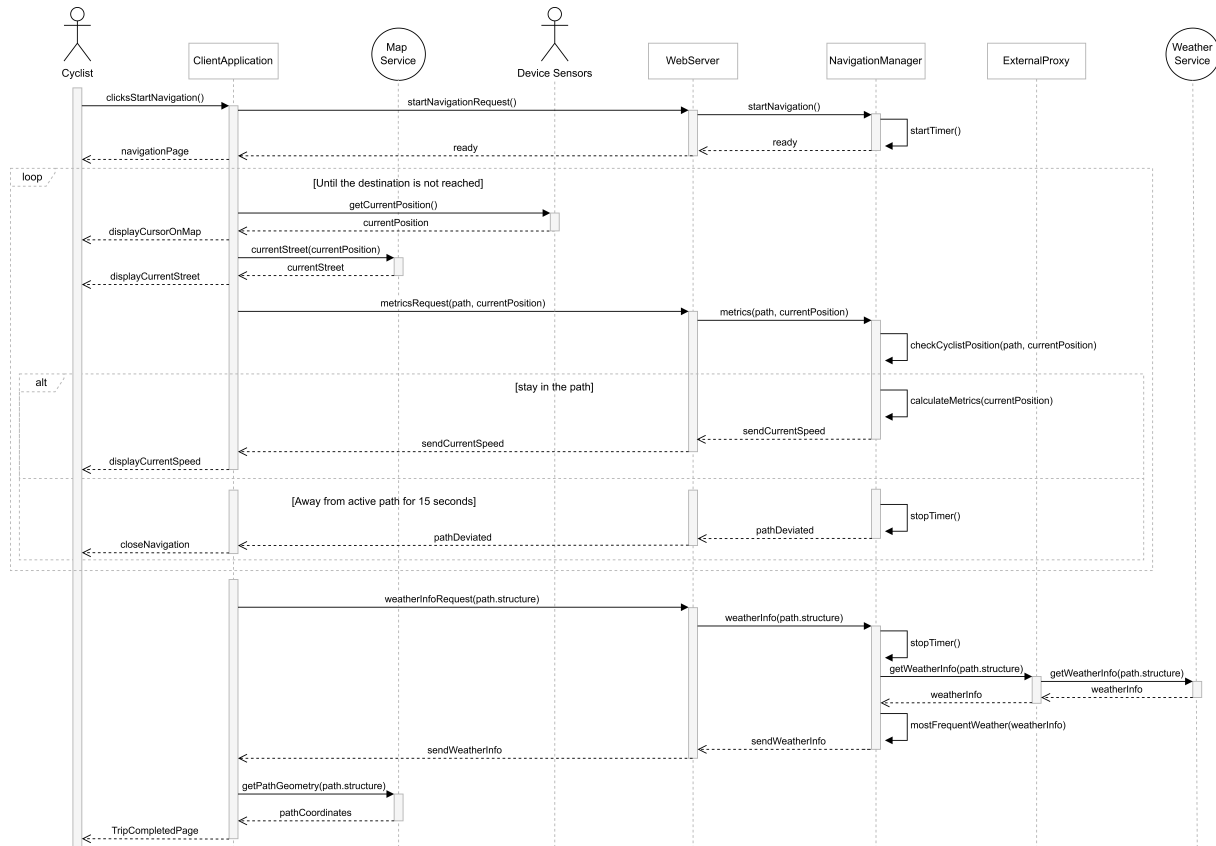
[SD4] Start Navigation Sequence Diagram

Figure 10: [SD4] Start Navigation Sequence Diagram

This sequence diagram describes the assisted navigation process for a cyclist, from the start until the completion of the trip. The Cyclist starts the navigation from the ClientApplication, which sends a start request to the WebServer. The NavigationManager initializes the navigation, starts a timer, and signals that the system is ready; the client then displays the navigation page. During the trip (loop block, until the destination is reached), the application retrieves the current position from the device sensors, updates the cursor on the map, displays the current road via the Map Service, and sends requests to the server to compute metrics based on the path and the current position. The NavigationManager checks whether the cyclist remains on the path and computes navigation metrics.

- If the cyclist stays on the path, the current speed is sent to the client and displayed to the user.
- If the cyclist deviates from the path for too long, a deviation is reported, the timer is stopped, and the navigation is terminated.

At the end of the trip, the client requests the weather from the weather service and the path geometry from the Map Service. The NavigationManager, through the ExternalProxy, queries the Weather Service, processes the result (the most frequent weather condition along the path), and sends it to the client, which displays the trip completion page with the final information.

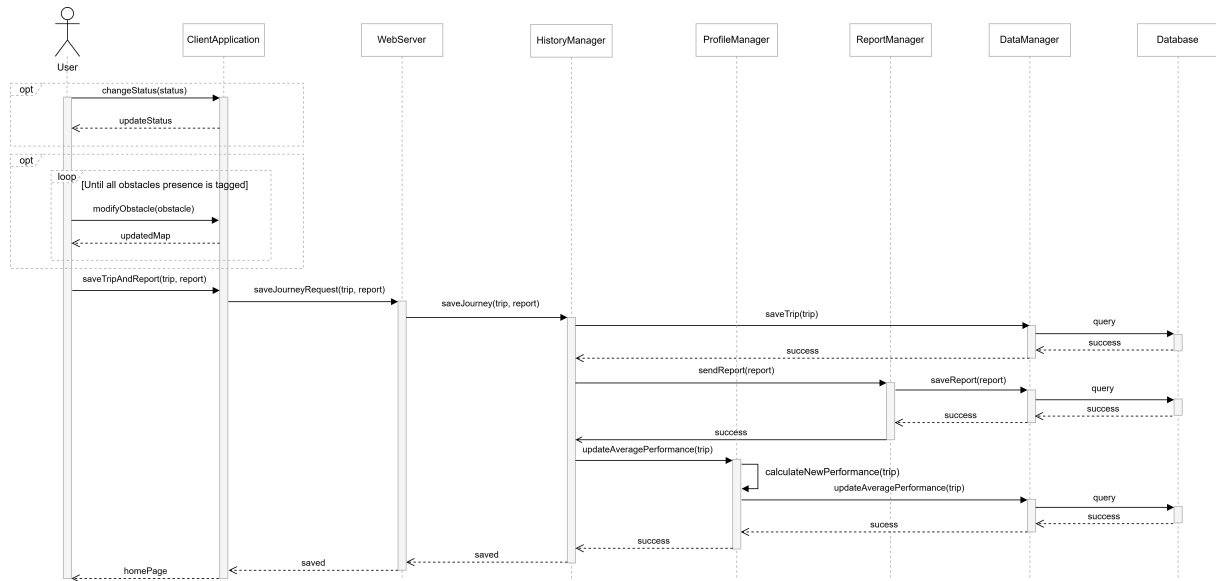
[SD5] Save Trip Sequence Diagram

Figure 11: [SD5] Save Trip Sequence Diagram

This sequence diagram illustrates the Save Trip workflow. The process could initiate with the user changes path status via the interface. An optional sequence, structured as a loop, allows the user to iteratively modify obstacle presence directly on the interface. The core persistence phase is triggered when the user saves the trip and report. The WebServer forwards this combined data to the HistoryManager, which coordinates the operation. First, it directly persists the trip data via the DataManager. Then, it delegates the report details to the ReportManager, which saves the report in the database via the DataManager. Finally, the HistoryManager interacts with the ProfileManager to calculate and update the user's average performance statistics. Upon the successful completion of this chain of operations, the success response propagates back through the system, and the user is redirected to the homepage.

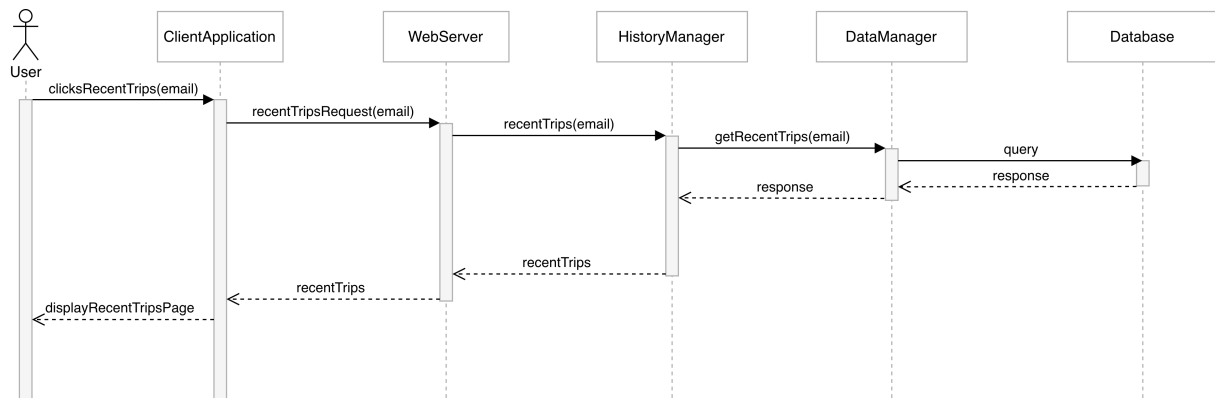
[SD6] View Trips Sequence Diagram

Figure 12: [SD6] View Trips Sequence Diagram

This sequence diagram shows the process of displaying the trips performed by the user in the dedicated section of the application, called Recent Trips. Initially, the user clicks on the Recent Trips option in the interface. This action triggers the ClientApplication to send a request containing the user identifier to the WebServer. The WebServer then forwards the request to the HistoryManager, which is responsible for handling the retrieval of the user's travel history. The HistoryManager contacts the DataManager in order to obtain the list of recent trips associated with the given user. The DataManager queries the Database and retrieves the corresponding data. The response is then propagated back from the Database to the DataManager and subsequently to the HistoryManager. Once the HistoryManager has received the list of recent trips, this information is sent back through the WebServer to the ClientApplication. Finally, the ClientApplication displays the Recent Trips page to the user, presenting the retrieved trips in the user interface.

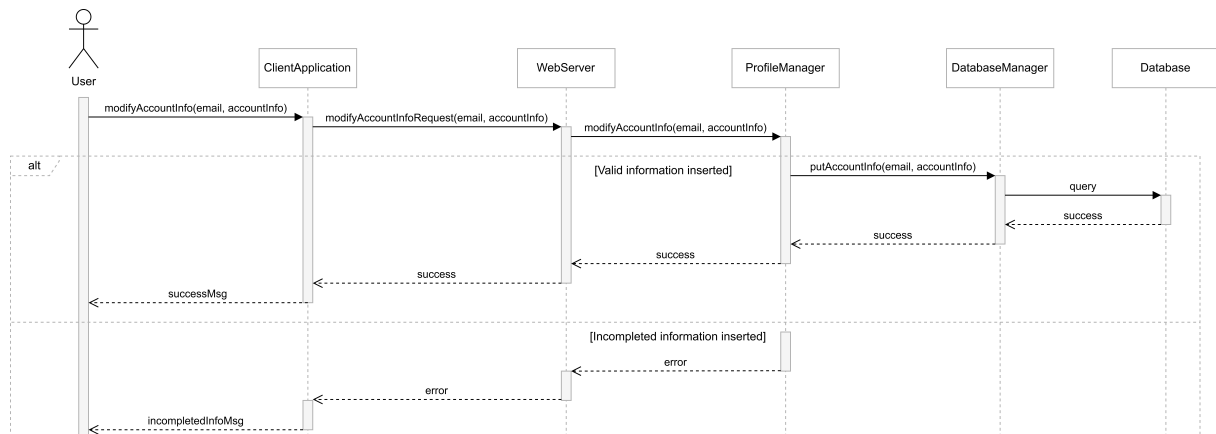
[SD7] Edit Profile Sequence Diagram

Figure 13: [SD7] Edit Profile Sequence Diagram

This sequence diagram describes the process of updating user profile information. The User sends the update request through the ClientApplication, which forwards it to the WebServer and then to the ProfileManager. The ProfileManager validates the submitted data and, if valid, asks the DatabaseManager to update the information in the Database.

- If the information is valid, the update is successfully completed, a success response propagates back through the chain, and the ClientApplication displays a confirmation message to the user.
- If the information is incomplete, an error is returned, propagates back to the ClientApplication, and is shown to the user as an incomplete data message.

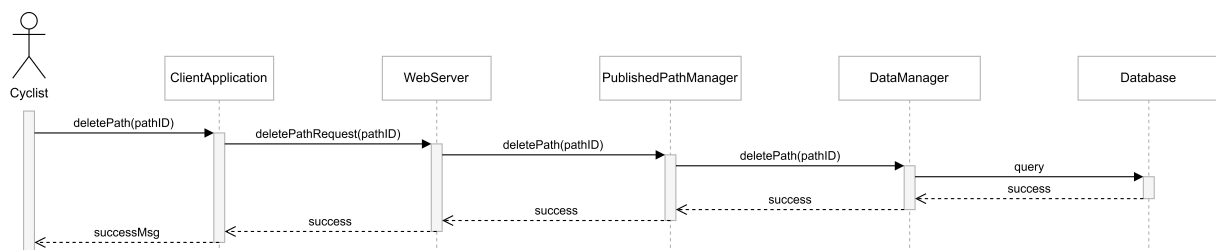
[SD8] Delete Path Sequence Diagram

Figure 14: [SD8] Delete Path Sequence Diagram

This sequence diagram illustrates the process of deleting a path initiated by a cyclist, showing how the request is propagated through the various architectural layers down to the Database. Once the deletion query is executed, the success message propagates back through the entire component chain to confirm the operation to the end user.

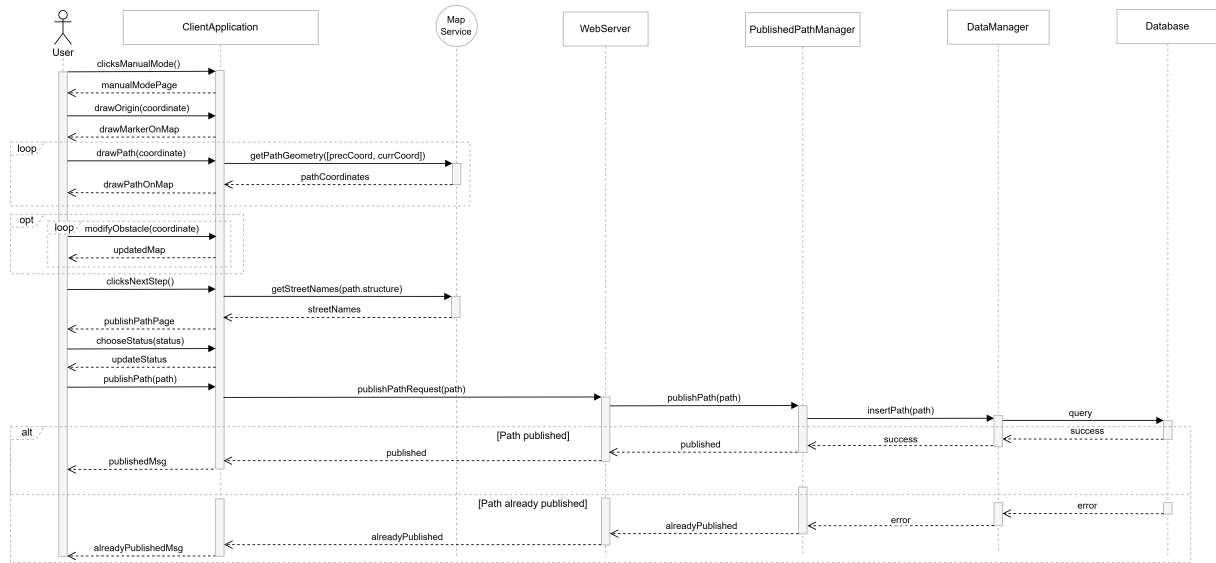
[SD9] Publish Path Sequence Diagram

Figure 15: [SD9] Publish Path Sequence Diagram

This sequence diagram illustrates the manual path creation workflow. The user draws the path on the ClientApplication, which iteratively queries the external MapService to retrieve path geometry and street names. Distinctly, the obstacle tagging process is handled locally by the interface: the user can modify obstacles receiving immediate visual updates without triggering external service calls. Once finalized, the publication request is propagated through the WebServer and PublishedPathManager to the DataManager for persistence. The outcome is handled in an alternative block:

- **Success:** The path is unique; the Database confirms insertion, and a success signal propagates back to notify the user.
- **Failure:** A duplicate path or error triggers a negative response, which is displayed to the user as an error message.

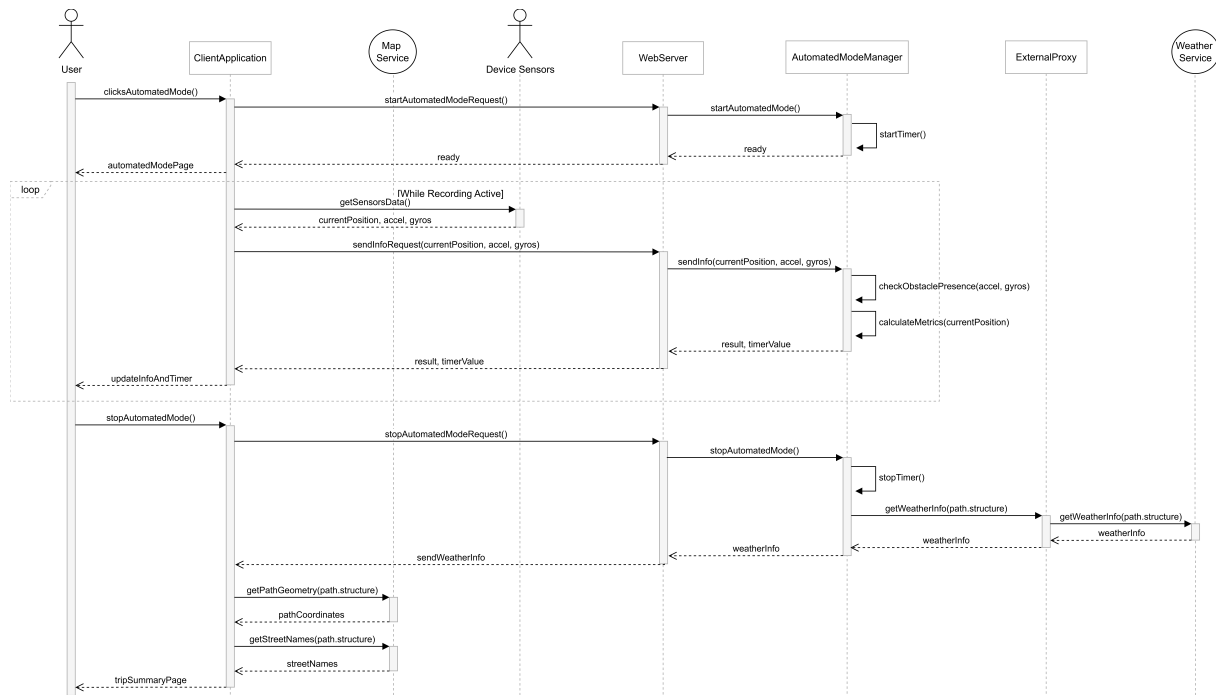
[SD10] Automated Mode Sequence Diagram

Figure 16: [SD10] Automated Mode Sequence Diagram

This sequence diagram describes the flow of the automatic activity recording mode. The User activates the automatic mode from the ClientApplication, which sends a start request to the WebServer. The WebServer delegates the operation to the AutomatedModeManager, which starts a timer and signals that the system is ready; the client then displays the automatic mode page. During the recording phase (loop block, while recording is active), the application collects data from the device sensors (gps, accelerometer, and gyroscope), sends these data to the AutomatedModeManager through the web server, and periodically receives the processing results and the elapsed time, which are displayed to the user in real time, with a small latency due to the data transmission time between client and server. The AutomatedModeManager checks for the presence of obstacles by analyzing accelerometer and gyroscope data, and computes activity metrics. When the user stops the automatic mode, the timer is stopped, the activity is marked as completed, and the client requests the weather from the Weather Service (through the web server and AutomatedModeManager) and the path geometry from the Map Service, so that they are already prepared for the display of the trip summary page, which is shown immediately afterward.

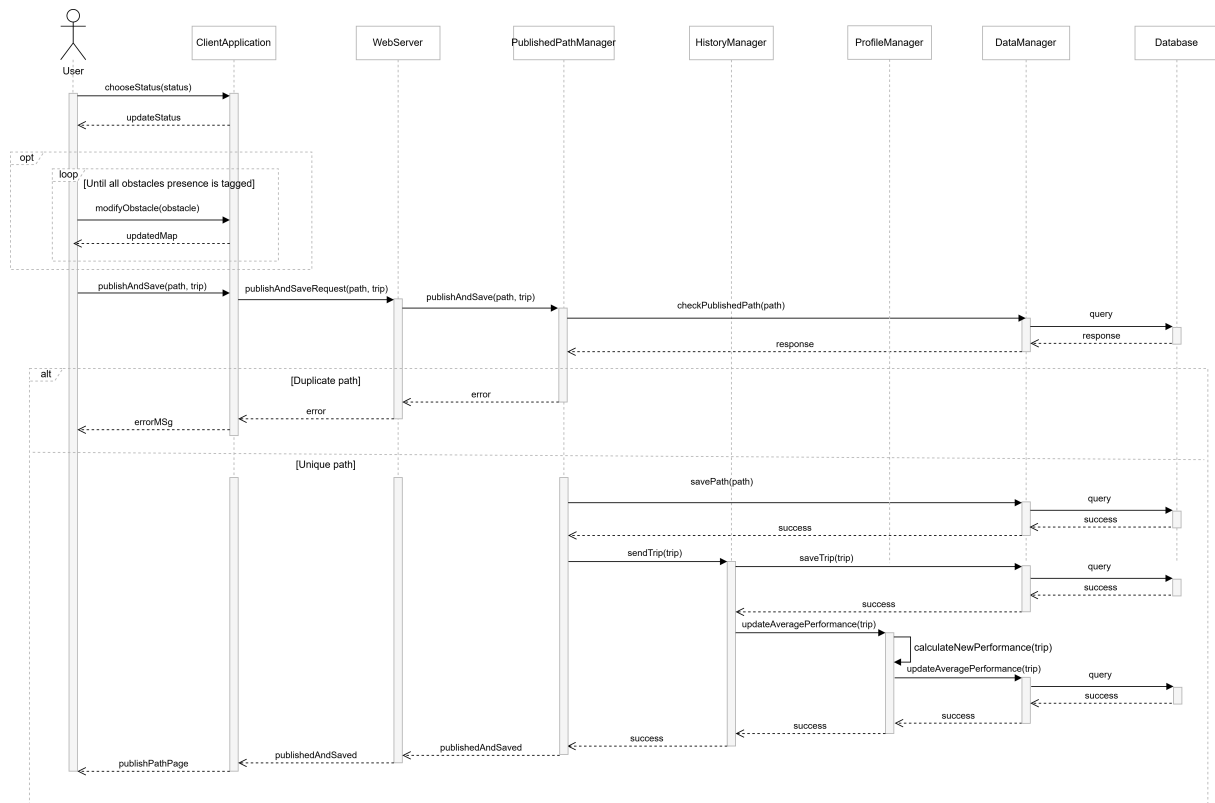
[SD11] Publish Recorded Path Sequence Diagram

Figure 17: [SD11] Publish Recorded Path Sequence Diagram

The process initiates with the user sets path status via the interface. An optional sequence, structured as a loop, allows the user to iteratively modify obstacle presence directly on the interface. Then, the user requesting to publish path and save a trip. This request is forwarded by the ClientApplication and WebServer to the PublishPathManager. Upon receiving the request, the PublishPathManager verifies the path's uniqueness by querying the DataManager.

An alternative frame handles two distinct scenarios:

- If the path already exists, the PublishPathManager returns an error, which propagates back through the system to the user as an error message.
- If the path is new, the system proceeds with a sequential persistence process. First, the PublishPathManager saves the path structure via the DataManager. Then, it delegates the trip details to the HistoryManager. The HistoryManager saves the trip data and subsequently triggers the ProfileManager to update the user's statistics. The ProfileManager calculates the new performance metrics and persists the updates via the DataManager. Only after all three operations (path, trip, and profile update) are confirmed, a success signal is returned and redirects user to publish path page.

3.5 Component Interface

This section describes all service offered by the interface of each component.

Business tier

3.5.1 AccountManager Interfaces

The AccountManager component exposes two interfaces: AccountServiceI, which is used by the Web-Server, and AccountManagerI, which is used by the TripManager.

AccountServiceI methods:

- **sendRegistration(user)**
It returns a confirmation link, indicating that the credentials have been successfully validated and triggering the start of the verification timer.
- **confirmRegistrationEmail()**
It returns a confirmation that the registration of guest in database was successful.
- **sendCredentials(email, pass)**
It returns a confirmation of the validity of the provided credentials by querying the database.
- **modifyAccountInfo(email, accountInfo)**
It returns a confirmation that the modification of account information in the database was successful.

AccountManagerI methods:

- **updateAveragePerformance(trip)**
It returns a confirmation that the update of the average performance metrics in the user's profile was successful.

3.5.2 PathManager Interfaces

The PathManager component exposes only the PathServiceI, which is used by the WebServer.

PathServiceI methods:

- **searchPath(origin, dest)**
It returns the top 5 paths based on the score.
- **deletePath(pathID)**
It returns a confirmation that the deletion from the database was successful.
- **publishPath(path)**
It returns a confirmation that the insertion in the database was successful.
- **startAutomatedMode()**
It returns a confirmation that the automated mode is started.
- **sendInfo(currentPosition, accel, gyros)**
It returns the processing result and the current timer value, derived after verifying the sensor data and calculating the trip metrics.
- **stopAutomatedMode()**
It returns a confirmation that the automated mode is stopped.
- **publishAndSave(path, trip)**
It returns a confirmation indicating that the path has been successfully published and the associated trip saved.

3.5.3 TripManager Interfaces

The TripManager component exposes two interfaces: TripServiceI, which is used by the WebServer, and TripManagerI, which is used by the PathManager.

TripServiceI methods:

- **startNavigation()**
It starts the internal navigation timer and returns a positive status indicating the system is ready to track the user.
- **metrics(path, currentPosition)**
It starts the cyclist position checking and the metrics calculation up to now, returns the current speed.
- **weatherInfo(List<coordinate>)**
It returns a collection of meteorological data, providing weather details for every coordinate in the provided list.
- **saveJourney(trip, report)**
It returns a signal confirming that both the trip data and the associated report have been successfully persisted in the system.
- **recentTrips(email)**
It returns the list of trips recently completed by the specified user, retrieved via the DataManager.

TripManagerI methods:

- **sendTrip(trip)**
It returns a confirmation indicating that the trip details have been successfully persisted by the DataManager.

3.5.4 ReportManager Interfaces

The ReportManager component exposes only the ReportManagerI, which is used by TripManager.

ReportManagerI methods:

- **sendReport(report)**
It returns a confirmation indicating that the report has been successfully saved.

3.5.5 DataManager Interfaces

The DataManager component exposes only the DataManagerI, which is used by all components of Business Tier: AccountManager, ReportManager, PathManager, TripManager.

DataManagerI methods:

- **checkCredentials(user)**
it returns a confirmation that the credentials are valid and that the user does not already exist in the database.
- **insert(user)**
it returns a confirmation that the insertion of the user's registration information into the database was successful.
- **checkCredentials(email, pass)**
it returns a confirmation that the provided email and password are correct and exist in the database.

- **getBestPaths(origin, dest)**
It returns the top 5 paths stored in the database that match the corresponding origin and destination, ranked based on their score.
- **saveTrip(trip)**
It returns a confirmation indicating that the trip details have been successfully persisted in the database.
- **saveReport(report)**
It returns a confirmation indicating that the report details have been successfully persisted in the database.
- **updateAveragePerformance(trip)**
It returns a confirmation indicating that the user's performance statistics have been recalculated based on the new trip details and successfully updated in the database.
- **getRecentTrips(email)**
It returns the raw data of the past trips performed by the user, fetched from the database.
- **putAccountInfo(email, accountInfo)**
It returns a success confirmation indicating that the account information associated with the specified user has been successfully updated in the database.
- **deletePath(pathID)**
It returns a confirmation indicating that the path associated with the specified ID has been successfully removed from the database.
- **insertPath(path)**
It returns a confirmation indicating that the new path details have been successfully inserted and persisted in the database.
- **checkPublishedPath(path)**
It returns the existence status of the path to prevent the creation of duplicate entries.
- **savePath(path)**
It returns a success signal acknowledging that the new path definition has been correctly stored.

Data tier

3.5.6 Database Interfaces

The Database component exposes the DBMS API, which is utilized by the DataManager to execute queries on the database and retrieve the results.

Web tier

3.5.7 WebServer Interfaces

The WebServer component exposes only the WebServerI, which is used by the ClientApplication. WebServerI methods:

- **sendRegistrationRequest(user)**
This request is forwarded to the RegistrationManager and returns a confirmation indicating that the system has successfully validated the credentials and sent the verification link.
- **confirmRegistrationEmailRequest()**
This request is forwarded to the RegistrationManager and returns a confirmation that the registration process was successful.
- **sendCredentialsRequest(email, pass)**
This request is forwarded to the AuthManager and returns a confirmation that all provided credential are correct and exist in the database.
- **searchPathRequest(origin, dest)**
This request is forwarded to the SearchPathManager and returns the top 5 best-scored paths with correspond origin and destination.
- **startNavigationRequest()**
This request is forwarded to the NavigationManager and returns a confirmation that the NavigationManager is ready.
- **metricRequest(path, currentPosition)**
This request is forwarded to the NavigationManager and returns the current street and the current speed.
- **weatherInfoRequest(List<coordinate>)**
This request is forwarded to the NavigationManager, which, through the ExternalProxy, requests weather information related to the path from the Weather Service. In particular, The Weather Service returns the weather condition that occurs most frequently among the provided coordinates.
- **saveJourneyRequest(trip, report)**
This request is forwarded to the HistoryManager and returns a confirmation that trip and report were saved in the database.
- **recentTripsRequest(email)**
This request is forwarded to the HistoryManager and returns the recent trips list.
- **modifyAccountInfoRequest(email, accountInfo)**
This request is forwarded to the ProfileManager and returns a confirmation that the update account information process was successful.

- **deletePathRequest(pathID)**
This request is forwarded to the PublishedPathManager and returns a confirmation that the deletion process was successful.
- **publishPathRequest(path)**
This request is forwarded to the PublishedPathManager and returns a confirmation that the publication process was successful.
- **startAutomatedModeRequest()**
This request is forwarded to the AutomatedModeManager and returns a confirmation that the AutomatedModeManager is ready.
- **sendInfoRequest(currentPosition, accel, gyros)**
This request is forwarded to the AutomatedModeManager and returns the total distance covered, the current and average speed, a boolean value represents the presence of an obstacle, and the timer value.
- **stopAutomatedModeRequest()**
This request is forwarded to the AutomatedModeManager and returns all the information related to the traveled path.
- **publishAndSaveRequest(path, trip)**
This request is forwarded to PublishPathManager and returns a confirmation that the new path being published and the trip details being securely stored in the database.

3.5.8 ExternalProxy Interfaces

The ExternalProxy exposes the ExternalServicesI interface, enabling components within the system to utilize the services provided by the MapService and WeatherService.

ExternalServicesI methods:

- **getWeatherInfo(List<coordinate>)**
This request is forwarded to the Weather Service and returns the weather information for all the coordinates provided.

Presentation tier

3.5.9 ClientApplication Interfaces

The ClientApplication component exposes the UserAppI, which is used by the cyclists.

UserAppI methods:

- **enterAccountInfo(user)**
It initiates the registration sequence by submitting the user's credentials. In the success scenario, this triggers the backend validation process which results in the system sending a verification email.
- **confirmEmail()**
it submits the user's registration details. Upon successful completion, the guest is redirected to the login page.
- **enterCredentials(email,pass)**
it submits the user's credentials. Upon successful completion, the guest is redirected to the account page.
- **enterDestination(dest)**
It returns the map on which the paths with the corresponding origin and destination are drawn.
- **enterOrigin(origin)**
It updates the internal state of the ClientApplication with the origin selected by the user, preparing the data for the navigation.
- **clicksStartNavigation()**
It triggers the navigation initialization sequence, resulting in the cyclist being redirected to the navigation interface once the system is ready.
- **chooseStatus(status)**
It updates the internal state of the ClientApplication with the status selected by the user, preparing the data for the trip report.
- **modifyObstacle(obstacle)**
It updates the local status of a specific obstacle and immediate refresh of the map display.
- **saveTripAndReport(trip, report)**
It initiates the final save operation for the completed trip and its associated report, ultimately redirecting the user to the home page.
- **clicksRecentTrips(email)**
It redirects user to recent trip page upon success.
- **modifyAccountInfo(email, accountInfo)**
It returns a confirmation indicating that the valid account information has been accepted and successfully updated in the database.
- **deletePath(pathID)**
It returns a confirmation indicating that the path associated with the specified ID has been successfully removed from the database.
- **clicksManualMode()**
It redirects the user to manual mode page.

- **drawOrigin(coordinate)**
It represents the user's interaction to define the starting point of the path on the map, which triggers the application to display a marker at the selected coordinate.
- **drawPath(coordinate)**
It allows the user to extend the path by adding new marker, triggers the application to calculate the connecting geometry and update the visual representation of the path on the map.
- **publishPath(path)**
It triggers the storage of the manually created path into the database via the PublishedPathManager, confirming the permanent addition of the new path.
- **clicksAutomatedMode()**
It triggers the start of the automated recording session, resulting in the system initialization and the transition to the automated mode page.
- **stopAutomatedMode()**
It ends the automated tracking process and redirects the user to the trip summary page to view the recorded path details.
- **publishAndSave(path, trip)**
It triggers a sequence that validates the path against existing records. Upon successful validation, it sequentially saves the path structure and the trip history to the database before confirming the operation to the user.
- **getMapData()**
This request is forwarded to the Map Service and returns the necessary metadata that allow the ClientApplication to initialize its local mapping engine and use graphical libraries.

External Services

3.5.10 MapService Interfaces

The external Map service exposes the MapServiceI, which is used by ExternalProxy.
MapServiceI methods:

- **currentStreet(currentPosition)**
It returns the name of the street corresponding to the current position, retrieved from the external Map Service.
- **getPathGeometry(List<coordinate>)**
It returns the detailed coordinates of the path, retrieved from the external Map Service, based on the provided list of coordinates of markers.

3.5.11 WeatherService Interfaces

The WeatherService exposes the WeatherServiceI, which is used by ExternalProxy.
WeatherServiceI methods:

- **getWeatherInfo(List<coordinate>)**
It returns a list of weather conditions, each one associated with one coordinate in the list passed as a parameter.

3.5.12 Sensors Interfaces

The Sensors exposes the SensorsI, which is used by ClientApplication.

Sensors methods:

- **getCurrentPosition()**
It returns the current location data detected by the device's GPS.
- **getSensorsData()**
It returns the real-time telemetry data, specifically the current position, accelerometer readings, and gyroscope values, retrieved directly from the device's sensors.

3.6 Selected Architectural Styles and Patterns

This section describes the architectural styles and design patterns selected for the Best Bike Paths (BBP) system, providing the rationale for their adoption in terms of scalability, security, and maintainability. The System is organized according to a robust and highly scalable architectural style for distributed applications: the Client-Server architecture with four tiers.

- **Presentation Tier**
It consists of application clients that make requests to the application server through the web server. It communicates with the map service and manages all graphical interactions with the user.
- **Web Tier**
This is the DMZ of the system. It consists of components that handle the interaction between the clients and the business logic. It includes the Web Server, which acts as the primary entry point for all incoming traffic, managing the logical routing of requests. Additionally, this tier hosts the External Proxy, a specialized component that manages outbound communication with third-party APIs (weather services). This ensures that the internal network remains isolated.
- **Business Tier**
It consists of components that provide the core business logic for the application. It processes the raw data coming from the Web Tier and coordinates data persistence toward the Data Tier.
- **Data Tier**
It consists of the database server and the DBMS. This layer handles the persistence of all information, which are accessed exclusively by components in the business tier through secure connections.

3.7 Other Design Decisions

- **Master-slave database architecture**
The Master-Slave architecture centralizes write operations on a single node (the master), while distributing read operations across multiple synchronized replicas (slaves) to optimize performance. The master is responsible for handling all write requests and propagating updates to the slaves to keep them consistent. This structure ensures service continuity even in the event of a failure of the primary server.
- **daily report merging**
Every day at midnight, the daily reports are merged, and the information of the corresponding paths is subsequently updated.

- **direct communication of the ClientApplication with the Map Service**

For performance reasons, crucial, for example, for communicating the name of the current street during navigation, the ClientApplication communicates directly with the Map Service, without passing through the Application Server. server.

- **No calculation requests are sent to the Map Service**

Only the map and the graphical libraries are requested from the Map Service; all calculations, such as checking whether the cyclist is on the route during navigation, are handled by the Application Server. This approach is adopted to remain as independent as possible from external services, requesting only the minimum necessary.

4 User Interface Design

Through the use of mockups, this chapter focuses on the graphical representation of the user profile-related screens, which were not detailed in the RASD

Home page

On the left, the *Home page* is shown from the user's perspective; on the right, from the guest's perspective.

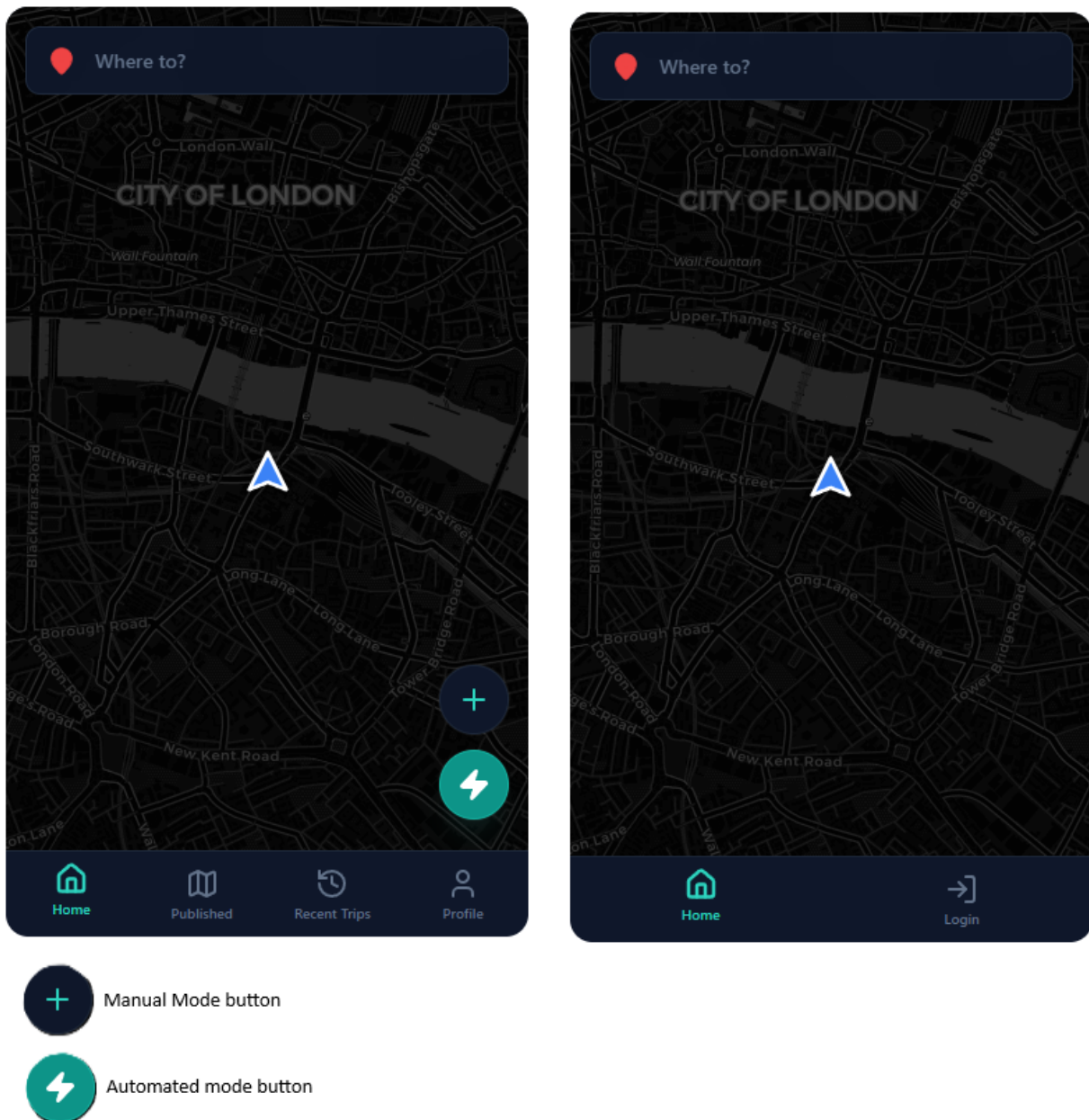


Figure 18: Home page

Profile

When the user clicks “Profile” on the navigation bar, the *Profile screen* is displayed. Here, the user can see all his statistics related to his trips and the number of path published by this account.

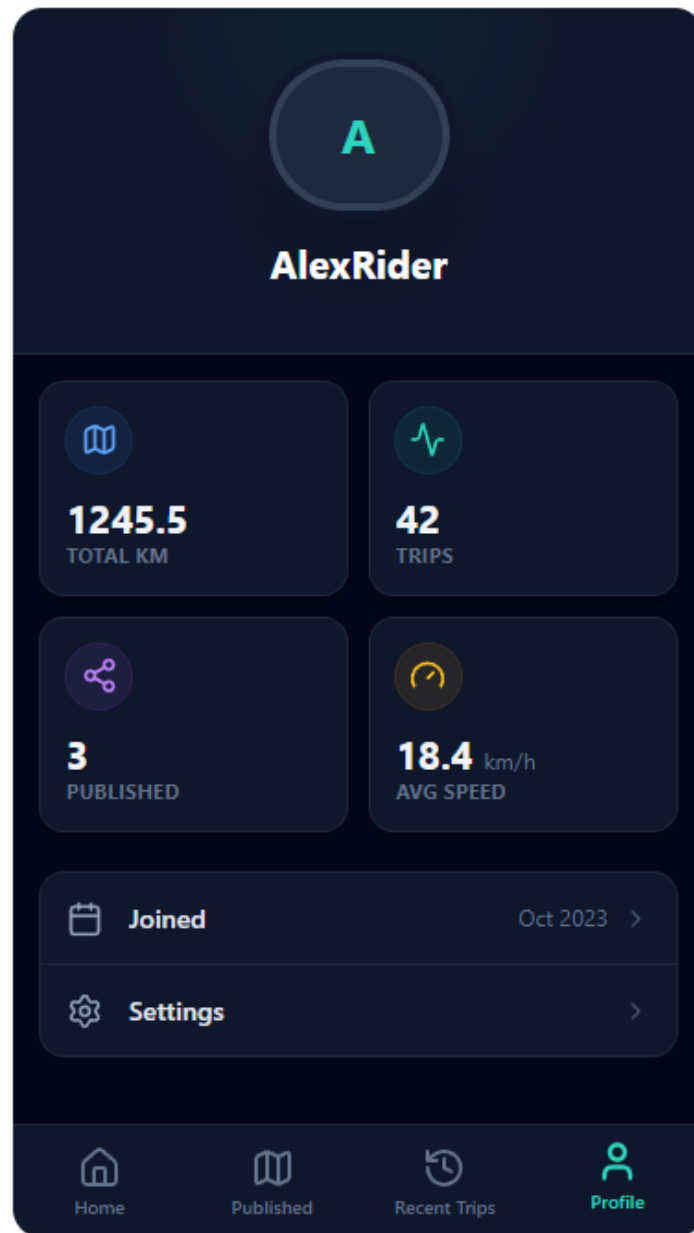


Figure 19: Profile screen

Edit profile screen

From the *Profile screen*, when the guest clicks “Settings”, this screen is displayed. Here, the user can modify his username and password, or delete the account.

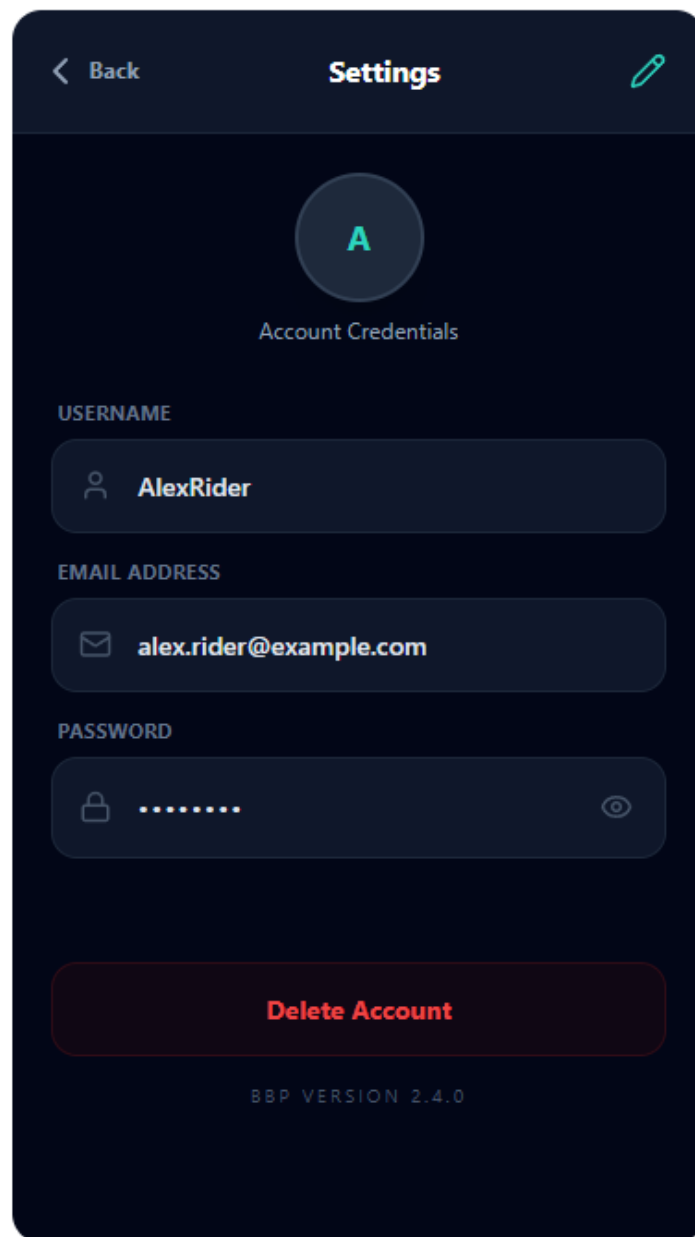


Figure 20: Edit profile screen

Login

When the guest clicks “Login” on the navigation bar, the *Login screen* is displayed. Here, the guest must enter their credentials in order to access their account and use all the services available to registered users (see the *Home page* on the left).

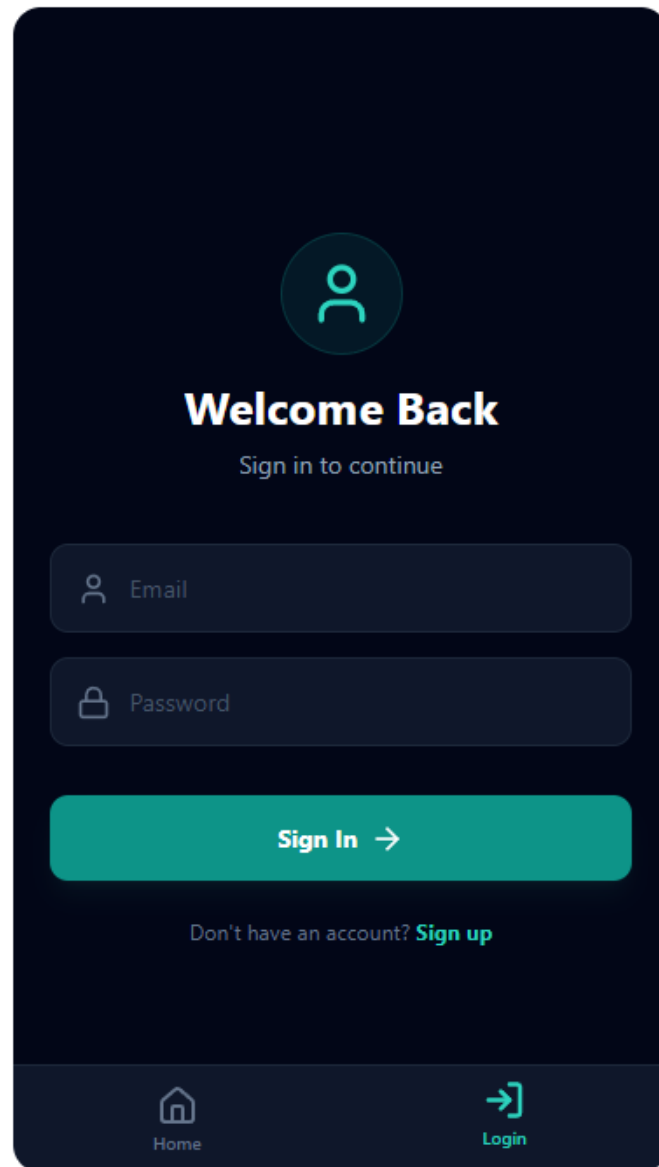


Figure 21: Login screen

Sign up

From the *Login screen*, when the guest clicks “Sign up”, this screen is displayed. Here, the user must enter their credentials and a username. Finally, the guest clicks “Sign up →”, the account is saved in the database, and the user is redirected to the *Profile screen*.

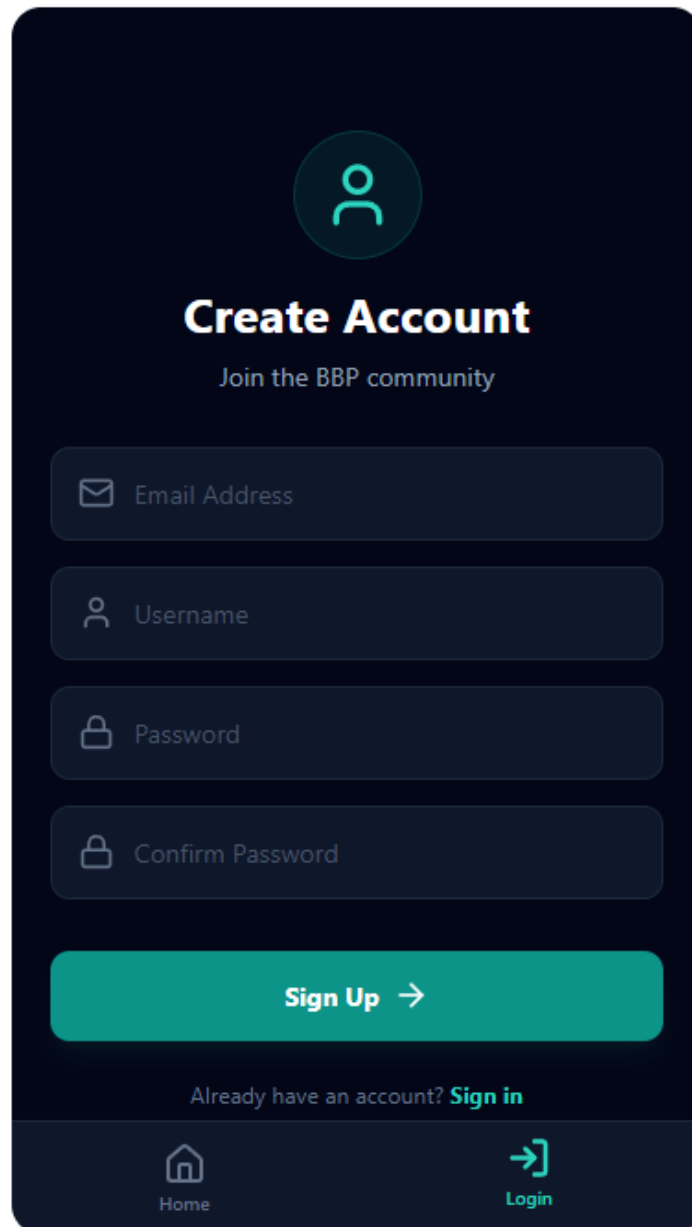
The image shows a mobile application screen for creating a new account. At the top, there is a circular profile icon placeholder. Below it, the title "Create Account" is displayed in a large, bold, white font, followed by the subtitle "Join the BBP community" in a smaller, lighter font. The form consists of four input fields: "Email Address" (with an envelope icon), "Username" (with a person icon), "Password" (with a lock icon), and "Confirm Password" (with a lock icon). All fields are currently empty. Below the input fields is a prominent teal button labeled "Sign Up →". At the bottom of the form area, there is a link that says "Already have an account? Sign in". The bottom of the screen features a dark navigation bar with two icons: a house icon labeled "Home" and a right-pointing arrow icon labeled "Login".

Figure 22: SignUp screen

5 Requirements Traceability

Internal Components

[C1] ClientApplication
[C2] WebServer
[C3] AccountManager
[C4] RegistrationManager
[C5] AuthManager
[C6] ProfileManager
[C7] PathManager
[C8] AutomatedModeManager
[C9] PublishedPathManager
[C10] PathSearchManager
[C11] TripManager
[C12] HistoryManager
[C13] NavigationManager
[C14] ReportManager
[C15] DataManager
[C16] Database
[C17] ExternalProxy

External Components

[E1] WeatherService
[E2] MapService

[R1]	The platform shall allow new users to register an account by providing personal information.
[C1]	ClientApplication
[C2]	WebServer
[C3]	AccountManager
[C4]	RegistrationManager
[C15]	DataManager
[C16]	Database

Table 1: Requirement R1 - Traceability

[R2]	The platform shall allow registered users to login by entering their registered email and password.
[C1]	ClientApplication
[C2]	WebServer
[C3]	AccountManager
[C5]	AuthManager
[C15]	DataManager
[C16]	Database

Table 2: Requirement R2 - Traceability

[R3]	The platform shall display an interactive map centered on the user's current location.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C9]	PublishedPathManager
[C10]	PathSearchManager
[C11]	TripManager
[C13]	NavigationManager
[E2]	MapService

Table 3: Requirement R3 - Traceability

[R4]	The platform shall allow users to input an "Origin" and "Destination" of the desired path.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C10]	PathSearchManager
[E2]	MapService

Table 4: Requirement R4 - Traceability

[R5]	The platform shall allow users to swap the origin and destination inputs.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C10]	PathSearchManager
[E2]	MapService

Table 5: Requirement R5- Traceability

[R6]	The platform shall calculate and display the top 5 paths options ranked by score.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C10]	PathSearchManager
[C15]	DataManager
[C16]	Database
[E2]	MapService

Table 6: Requirement R6 - Traceability

[R7]	The platform shall visualize the geometry, obstacles and scores of the top 5 paths on the map.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C10]	PathSearchManager
[C15]	DataManager
[C16]	Database
[E2]	MapService

Table 7: Requirement R7 - Traceability

[R8]	When the origin coincides with the current location, the platform shall allow users to start the navigation mode for the currently selected path.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C10]	PathSearchManager
[C11]	TripManager
[C13]	NavigationManager
[E2]	MapService

Table 8: Requirement R8 - Traceability

[R9]	When the navigation is started, the platform shall visualize the user's position in real-time.
[C1]	ClientApplication
[C2]	WebServer
[C11]	TripManager
[C13]	NavigationManager
[E2]	MapService

Table 9: Requirement R9 - Traceability

[R10]	When the navigation is started, the platform shall display a dynamic navigation overlay providing the name of the street being traversed and the estimated speed.
[C1]	ClientApplication
[C2]	WebServer
[C11]	TripManager
[C13]	NavigationManager
[E2]	MapService

Table 10: Requirement R10 - Traceability

[R11]	The platform shall allow users to interrupt the active navigation mode at any time.
[C1]	ClientApplication
[C2]	WebServer
[C11]	TripManager
[C13]	NavigationManager
[E2]	MapService

Table 11: Requirement R11 - Traceability

[R12]	When the navigation is completed, the platform shall present to registered users a summary screen containing statistics and the map of the path taken.
[C1]	ClientApplication
[C2]	WebServer
[C11]	TripManager
[C12]	HistoryManager
[C13]	NavigationManager
[C17]	ExternalProxy
[E1]	WeatherService
[E2]	MapService

Table 12: Requirement R12 - Traceability

[R13]	When the navigation is completed, the platform shall allow registered users to modify the path status and the obstacles.
[C1]	ClientApplication
[C2]	WebServer
[C11]	TripManager
[C12]	HistoryManager
[C13]	NavigationManager
[C14]	ReportManager
[E2]	MapService

Table 13: Requirement R13 - Traceability

[R14]	The platform shall allow registered users to start an active recording session.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C8]	AutomatedModeManager
[C10]	PathSearchManager
[E2]	MapService

Table 14: Requirement R14 - Traceability

[R15]	The platform shall utilize device sensors to automatically detect obstacles
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C8]	AutomatedModeManager
[E2]	MapService

Table 15: Requirement R15 - Traceability

[R16]	The platform shall display real-time metrics during the trip, including duration, current speed and distance traveled
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C8]	AutomatedModeManager
[C11]	TripManager
[C13]	NavigationManager
[E2]	MapService

Table 16: Requirement R16 - Traceability

[R17]	The platform shall allow registered users to stop the recording and immediately transition to the summary screen to review the captured data.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C8]	AutomatedModeManager
[C11]	TripManager
[C12]	HistoryManager
[C17]	ExternalProxy
[E1]	WeatherService
[E2]	MapService

Table 17: Requirement R17 - Traceability

[R18]	The platform shall allow registered users to modify status and obstacles in the captured data summary
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C8]	AutomatedModeManager
[C11]	TripManager
[C12]	HistoryManager
[E2]	MapService

Table 18: Requirement R18 - Traceability

[R19]	The platform shall allow registered users to publish the captured path along with its data.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C8]	AutomatedModeManager
[C11]	TripManager
[C12]	HistoryManager
[C15]	DataManager
[C16]	Database
[E2]	MapService

Table 19: Requirement R19 - Traceability

[R20]	The platform shall allow registered users to manually draw the path on the map.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C9]	PublishedPathManager
[E2]	MapService

Table 20: Requirement R20 - Traceability

[R21]	The platform shall allow registered users to manually place obstacle markers along the drawn path.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C9]	PublishedPathManager
[E2]	MapService

Table 21: Requirement R21 - Traceability

[R22]	The platform shall allow registered users to set the status of the drawn path.
[C1]	MobileApplication
[C2]	WebServer
[C7]	PathManager
[C9]	PublishedPathManager
[E2]	MapService

Table 22: Requirement R22 - Traceability

[R23]	The platform shall allow registered users to publish the drawn path along with its data.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C9]	PublishedPathManager
[C15]	DataManager
[C16]	Database
[C17]	ExternalProxy
[E1]	WeatherService
[E2]	MapService

Table 23: Requirement R23 - Traceability

[R24]	The platform shall display a dashboard listing the user's historical trips and their data, sorted chronologically.
[C1]	ClientApplication
[C2]	WebServer
[C11]	TripManager
[C12]	HistoryManager
[C15]	DataManager
[C16]	Database
[C17]	ExternalProxy
[E1]	WeatherService
[E2]	MapService

Table 24: Requirement R24 - Traceability

[R25]	The platform shall allow registered users to view a list of paths published by them.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C9]	PublishedPathManager
[C15]	DataManager
[C16]	Database
[E2]	MapService

Table 25: Requirement R25 - Traceability

[R26]	The platform shall allow registered users to delete any of their published paths.
[C1]	ClientApplication
[C2]	WebServer
[C7]	PathManager
[C9]	PublishedPathManager
[C15]	DataManager
[C16]	Database
[E2]	MapService

Table 26: Requirement R26 - Traceability

[R27]	The platform shall display a user profile screen summarizing the main aggregate statistics.
[C1]	ClientApplication
[C2]	WebServer
[C6]	ProfileManager

Table 27: Requirement R27 - Traceability

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17	E1	E2
R1	X	X	X	X											X	X			
R2	X	X	X		X										X	X			
R3	X	X					X		X	X	X		X						X
R4	X	X					X			X									X
R5	X	X					X			X									X
R6	X	X					X			X					X	X			X
R7	X	X					X			X					X	X			X
R8	X	X					X			X	X		X						X
R9	X	X									X		X						X
R10	X	X									X		X						X
R11	X	X									X		X						X
R12	X	X									X	X	X				X	X	X
R13	X	X									X	X	X	X					X
R14	X	X					X	X		X									X
R15	X	X					X	X											X
R16	X	X					X	X			X		X						X
R17	X	X					X	X			X	X					X	X	X
R18	X	X					X	X			X	X							X
R19	X	X					X	X			X	X			X	X			X
R20	X	X					X		X										X
R21	X	X					X		X										X
R22	X	X					X		X										X
R23	X	X					X		X						X	X	X	X	X
R24	X	X									X	X			X	X	X	X	X
R25	X	X					X		X						X	X			X
R26	X	X					X		X						X	X			X
R27	X	X				X													

Table 28: Requirements Traceability Matrix

6 Implementation, Integration, and Test Plan

6.1 Plan Overview

The four tiers that constitute the System (Client Application, Web Server, Application Server, and Data Manager) represent distinct functional layers that can be implemented primarily in parallel. By defining the interfaces early in the process such as the `AccountServiceI`, `TripServiceI`, and `PathServiceI` exposed by the Web Server development teams can work simultaneously on the client-side presentation and the server-side logic, integrating them continuously.

The Application Server layer encapsulates the core business logic and presents internal dependencies that require a more structured approach. For this central component, a bottom-up approach will be adopted. This strategy dictates starting with the implementation of independent and low-level modules before moving to those that orchestrate complex workflows.

The following diagram details the dependencies between the various components of the Application Server, including the relationships with the proxy component responsible for external data retrieval. In this representation, a hierarchical convention applies: if a component depicts a dependency on another component, it implies that all its internal sub-components share this dependency. Conversely, if a dependency is exclusive to a specific sub-component, a direct link is established solely to that element.

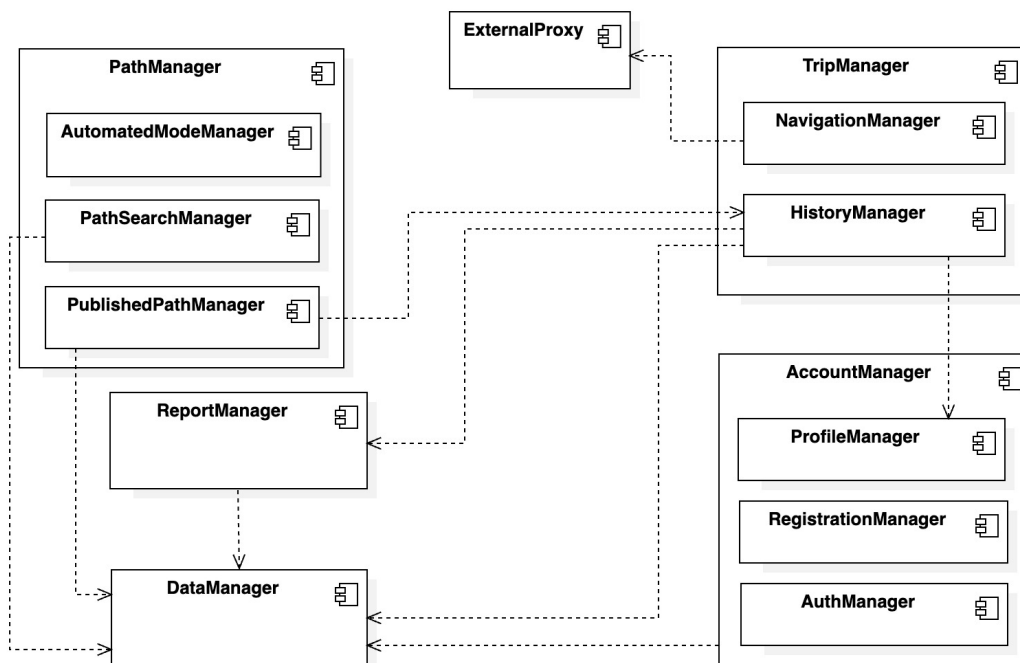


Figure 23: Dependencies Diagram

6.2 Implementation Plan

The following plan focuses strictly on the Application Server components and the External Proxy, where the dependency chain dictates a sequential implementation order.

1. **DataManager:** This is the first module to be implemented. Indeed, all the other components of the Application Server rely on it to communicate with the Database. Therefore, it is a module of crucial importance but characterized by a low level of functional complexity as it primarily implements the queries and CRUD operations.
2. **ExternalProxy:** This component is developed in parallel with or immediately after the DataManager. It is responsible for abstracting the communication with external systems, specifically the WeatherService. Its early implementation is essential to provide the necessary weather data of the trips (managed by TripManager).
3. **AccountManager:** The implementation of this module follows. It comprises the AuthManager, RegistrationManager, and ProfileManager. Its development is prioritized because it establishes the user context required by the subsequent components. Specifically, the ProfileManager exposes the interfaces necessary to manage user data.
4. **ReportManager** This component acts independently of the user logic and can be implemented at this stage. It manages the reports and implements a specific aggregation logic to merge multiple reports generated by different users for the same path within the same day. It interacts directly with the DataManager for persistence. Its completion is a prerequisite for the TripManager, as the reporting logic is triggered by completed trips.
5. **TripManager:** This component is implemented at this stage. It contains the NavigationManager and the HistoryManager. Its development relies on the completion of both the AccountManager and the ReportManager. The HistoryManager explicitly depends on the ProfileManager to update user statistics upon trip completion, and on ReportManager to submit user-generated reports. The NavigationManager depends on the ExternalProxy for the weather service API calls.
6. **PathManager:** This is the final module to be implemented. It contains the PathSearchManager, AutomatedModeManager, and PublishedPathManager. This component is placed last in the sequence due to a specific functional dependency: the PublishedPathManager requires access to the HistoryManager (part of the TripManager) to save the trip details into the user's history once an automated path acquisition is concluded.

6.3 Component Integration and Testing

This section describes the integration plan for the system components. Each component must first undergo Unit Testing in isolation. The integration process then occurs incrementally following a Bottom-Up approach: as soon as a component at a lower dependency level is stable, it is integrated with the components of the upper level.

To facilitate this process before the complete system is available, we will utilize Test Drivers: software modules used to invoke the component under test, simulating the requests that would normally come from the Web Server or other managers. The integration sequence is organized into four logical stages:

Stage 1: Data Persistence and External Services

In the first stage, we focus on the integration of the DataManager and the ExternalProxy, which act as the foundational layer of the Application Server. Since these components interact directly with the Database and external services respectively, their stability is a prerequisite for the entire system. Because the higher-level managers (such as the AccountManager or TripManager) are not yet developed at this stage, we utilize Test Drivers to simulate the invocation of these components:

- A Persistence Driver is developed to simulate the queries that will eventually be performed by the business logic, verifying the DataManager's ability to execute CRUD operations correctly.
- A Proxy Driver is created to simulate requests for weather data, ensuring that the ExternalProxy correctly interfaces with the WeatherService API and parses its response.

The following diagram illustrates this initial integration step, showing the drivers stimulating the base components in isolation.

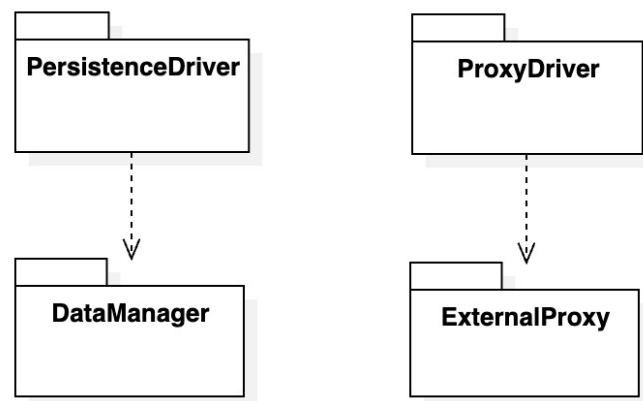


Figure 24: Stage 1

Stage 2: Independent Business Managers

The second stage involves the integration of the AccountManager and the ReportManager. These components are built upon the persistence layer validated in the previous stage. Since these managers are functionally independent of each other but rely on the DataManager for data storage and retrieval, they are integrated in parallel:

- The AccountManager is connected to the DataManager to verify that user registration, authentication, and profile updates are correctly persisted in the database.
- The ReportManager is integrated with the DataManager to ensure that infrastructure reports are correctly stored and that the aggregation (merging) logic successfully retrieves existing reports from the database.

To simulate the requests that would normally originate from the Web Server or the TripManager, specific Drivers are employed to invoke the methods exposed by these components.

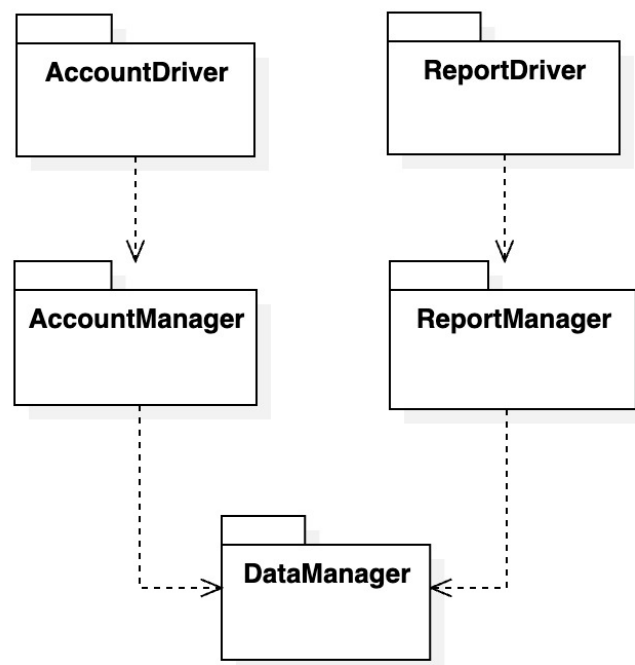


Figure 25: Stage 2

Stage 3: Trip Orchestration Logic

In the third stage, we integrate the TripManager, which encompasses the NavigationManager and HistoryManager. The TripManager is integrated on top of the components validated in Stage 2. In particular, the integration tests focus on verifying the following interactions:

- **Navigation Monitoring:** Verifying the core logic of the NavigationManager that, upon receiving real-time GPS updates, checks the user's adherence to the calculated path and computes real-time session statistics
- **User Statistics Update:** Verifying that the HistoryManager correctly invokes the ProfileManager (component of the AccountManager) exclusively upon trip completion to update the user's aggregated statistics.
- **Report Submission:** Ensuring that the HistoryManager correctly forwards path data to the ReportManager when a trip is finalized.
- **Trip Persistence:** Verifying that the trip session data is correctly stored via the DataManager.

A Trip Driver is used to simulate the full lifecycle of a trip (start, location updates, stop), ensuring that the manager correctly coordinates these diverse dependencies.

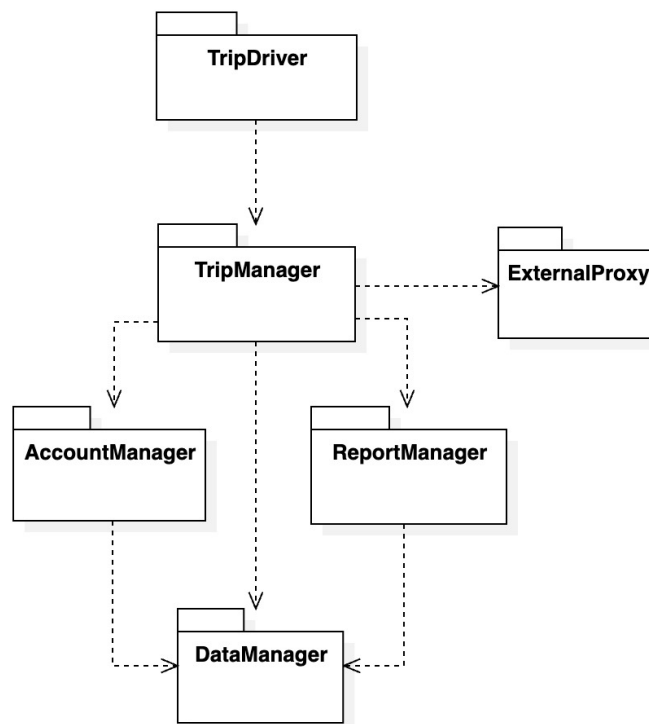


Figure 26: Stage 3

Stage 4: Final Application Server Integration

The final stage completes the Application Server integration by adding the PathManager. This component is responsible for the core routing algorithms and relies on internal trip management subsystem. In this phase, we integrate the PathManager with the components validated in Stage 3. While covering the entire operational scope of the component, the integration process places particular emphasis on the following critical interactions:

- **Automated mode trip Logic Integration:** Verifying that the PublishedPathManager can correctly access the TripManager (specifically the HistoryManager) to save the details of completed automated trip into the user's history.
- **Full System Flow:** Executing scenarios where the Path Driver triggers the complete operational chain: "Search Path (PathManager) -> Start Navigation (TripManager) -> Complete Trip -> Save to History & Submit Report (TripManager) -> Update User Statistics (ProfileManager)". This verifies that the path object provided by the search logic is correctly interpreted by the navigation engine and that all side effects, persisting the trip history, submitting the report for merging, and updating user statistics are successfully committed.

At the conclusion of this stage, the backend logic is fully integrated. The Test Drivers can then be replaced by the actual Web Server and Client Application for the System Testing phase.

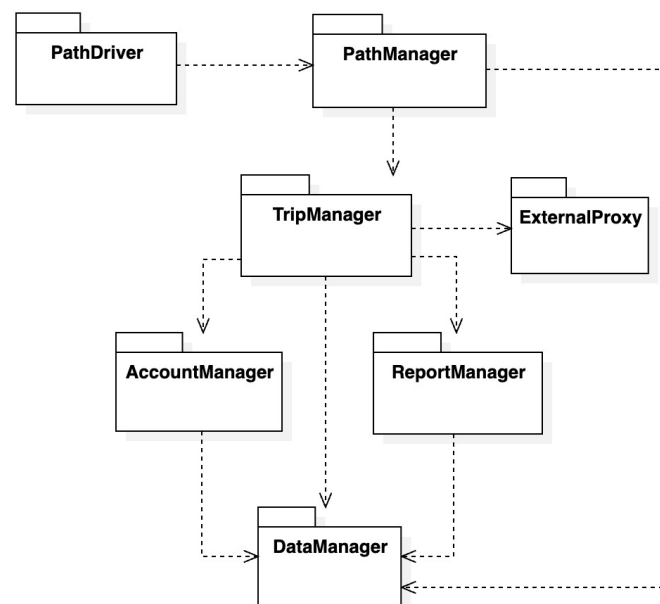


Figure 27: Stage 4

6.4 System Testing

Once all the Application Server components have been integrated and the mock drivers replaced by the actual Web Server and Client interfaces, System Testing begins. This process aims at verifying both functional and non-functional requirements and must take place in a testing environment that mirrors the production configuration as closely as possible. Specifically, the system will be subjected to the following testing categories:

- **Functional Testing:** It ensures that the requirements and specifications defined in the RASD are properly satisfied by the System. In this phase, we verify the complete end-to-end user flows.
- **Performance Testing:** The main purpose is to identify and eliminate performance bottlenecks affecting response time and throughput.
- **Load Testing:** It aims at detecting bugs such as memory leaks and buffer overflows, identifying the maximum operating capacity of the application.
- **Stress Testing:** It verifies the stability and reliability of the software application under extreme conditions. The goal is to ensure robustness and proper error handling.

7 Use of Generative AI

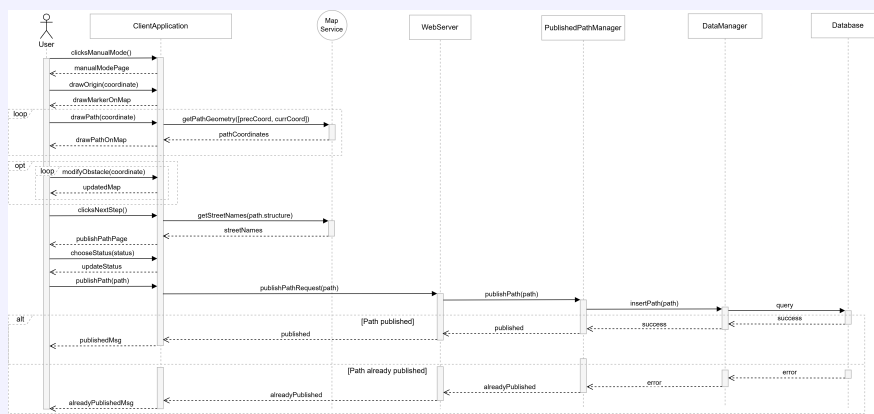
The application of Generative AI has shifted significantly between the two main phases of the project. While in the RASD phase, models were primarily used for translating textual use cases into high-fidelity UI mockups, in the DD, the focus moved towards textual reliability and technical precision.

In this phase, Google Gemini was not used to invent features, but rather to act as a rigorous technical writer. The primary goal was to ensure that the written documentation perfectly matched the architectural decisions made by the team.

7.1 Example of Interaction: Publish Path Sequence Diagram Description

The following examples illustrate the practical interaction with Gemini used to draft the documentation. In this workflow, the Sequence Diagrams were uploaded as visual inputs; the model analyzed the architectural logic depicted in the images and generated the corresponding textual descriptions presented in this document.

Prompt Give me a short description of this sequence diagram



Gemini The User activates manual mode from the ClientApplication, which displays the dedicated page and allows the user to draw the origin and the path directly on the map. During the drawing phase (the loop block), the client requests the path geometry from the WebServer; the request is forwarded to the PublishedPathManager, which queries the Map Service through the Proxy to obtain the correct path coordinates, which are then displayed on the map. Optionally (the opt block), the user can modify obstacles along the path, updating the map in real time.

When the user decides to publish the path, the client sends a publishPath request to the server. In the alt block:

If the path has not been published yet, the DataManager stores it in the Database, the operation succeeds, and the user receives a confirmation message.

If the path has already been published, an error is returned and the client shows a “path already published” message.

The diagram highlights the complete flow of path drawing, validation, and publication

The textual descriptions generated by Gemini accurately captured the logical flow and the sequence of messages depicted in the diagrams. However, these outputs were utilized primarily as initial drafts. The development team subsequently reviewed and reworked the content independently to ensure it perfectly aligned with the specific design nuances. The AI was leveraged mainly to establish a high standard of English technical writing and grammatical correctness, acting as an advanced drafting assistant rather than an autonomous author.

7.2 Conclusion

Although Gemini significantly accelerated the drafting phase, the final responsibility for the accuracy of the system specifications lies entirely with the development team. Every line of text was manually verified to strictly adhere to the project's requirements.

8 Effort Spent

Andrea Ferritti

Section	Hours
Introduction	11
Architectural Design	32
User Interface Design	23
Requirements Traceability	5
Implementation, Integration, and Test Plan	5
Use of Generative AI	2

Table 29: Effort - Andrea Ferritti

Jieke Lin

Section	Hours
Introduction	13
Architectural Design	28
User Interface Design	25
Requirements Traceability	3
Implementation, Integration, and Test Plan	3
Use of Generative AI	3

Table 30: Effort - Jieke Lin

Tommaso Tosca

Section	Hours
Introduction	12
Architectural Design	31
User Interface Design	25
Requirements Traceability	3
Implementation, Integration, and Test Plan	3
Use of Generative AI	2

Table 31: Effort - Tommaso Tosca